

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI TẬP
MÔN HỌC: KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM
Bài tập: TIỂU LUẬN FINAL

Họ và tên: Đỗ Ngọc Lâm
Mã sinh viên: B22DCCN476

Giảng viên: PGS. TS. Trần Đình Quế

Hà Nội, 2026

MỤC LỤC

1. Từ Monolithic đến Microservices và DDD	8
1.1. Giới thiệu Monolithic Architecture	8
1.1.1. Khái niệm	8
1.1.2. Cấu trúc điển hình	8
1.1.3. Ví dụ thực thể	9
1.1.4. Nhược điểm chi tiết	9
1.1.5. Khi nào nên dùng Monolithic	9
1.2. Microservices Architecture	10
1.2.1. Khái niệm	10
1.2.2. Đặc điểm	13
1.2.3. So sánh Monolithic và Microservices	14
1.2.4. Ưu điểm	15
1.2.5. Nhược điểm	15
1.2.6. Nguyên tắc thiết kế	15
1.3. Domain Driven Design (DDD)	16
1.3.1. Mục tiêu	16
1.3.2. Các khái niệm cốt lõi	16
1.3.3. Context Map	16
1.3.4. DDD trong Microservices	16
1.4. Case Study: Healthcare (Luyện Decomposition)	17
1.4.1. Mô tả bài toán	17
1.4.2. Bước 1: Xác định Domain	17
1.4.3. Bước 2: Xác định Bounded Context	17
1.4.4. Bước 3: Phân rã thành Microservices	17
1.4.5. Bước 4: Xác định quan hệ	17
1.4.6. Ví dụ API	18
1.5. Kết luận	18
2. Phát triển Hệ E-Commerce Microservices	23
2.1. Xác định yêu cầu	23
2.1.1. Functional Requirements	23
2.1.2. Non-functional Requirements	24
2.2. Phân rã hệ thống theo DDD	24

2.2.1.	Bounded Context	24
2.2.2.	Nguyên tắc.....	25
2.3.	Thiết kế Product Service (Django).....	25
2.3.1.	Phân loại sản phẩm	25
2.3.2.	Model tổng quát.....	25
2.3.3.	Chi tiết theo domain.....	25
2.3.4.	API.....	26
2.4.	Thiết kế User Service (Django).....	26
2.4.1.	Phân loại người dùng	26
2.4.2.	Model.....	26
2.4.3.	Phân quyền (RBAC)	26
2.4.4.	API.....	27
2.5.	Thiết kế Cart Service (Django).....	27
2.5.1.	Model.....	27
2.5.2.	Logic	28
2.5.3.	API.....	30
2.6.	Thiết kế Order Service (Django).....	31
2.6.1.	Model.....	31
2.6.2.	Workflow	33
2.7.	Thiết kế Payment Service (Django).....	35
2.7.1.	Model.....	35
2.7.2.	Trạng thái	37
2.7.3.	API.....	39
2.8.	Thiết kế Shipping Service (Django)	40
2.8.1.	Model.....	40
2.8.2.	Trạng thái	42
2.8.3.	API.....	44
2.9.	Luồng hệ thống tổng thể	44
2.10.	Hướng dẫn thực hành	50
2.10.1.	Mục tiêu.....	50
2.10.3.	Mapping Class Diagram sang Database	53
2.10.4.	Thiết kế Database cho từng Service	55
2.10.5.	So sánh MySQL vs PostgreSQL	58

2.10.6.	Bài tập	58
2.10.7.	Checklist đánh giá.....	59
3.	AI Service cho tư vấn sản phẩm	63
3.1.	Mục tiêu.....	63
3.2.	Kiến trúc AI Service	63
3.3.	Thu thập dữ liệu	63
3.3.1.	User Behavior Data	64
3.3.2.	Ví dụ dataset	64
3.4.	Mô hình LSTM (Sequence Modeling)	64
3.4.1.	Ý tưởng.....	64
3.4.2.	Model chi tiết	65
3.5.	Knowled Graph với Neo4j	65
3.5.1.	Mô hình đồ thị	65
3.5.2.	Ví dụ Cypher	66
3.5.3.	Truy vấn gợi ý.....	67
3.6.	RAG (Retrieval-Augmented Generation).....	68
3.6.1.	Pipeline.....	68
3.6.2.	Vector Database	69
3.6.3.	Ví dụ	69
3.7.	Kết hợp HybridModel.....	70
3.8.	Hai dạng AI Service.....	71
3.8.1.	Recommendation List	71
3.8.2.	Chatbot tư vấn.....	71
3.9.	Triển khai AI Service	72
3.9.1.	Tech stack	72
3.10.	Bài tập.....	74
3.11.	Checklist đánh giá	76
3.12.	Kết luận	77
4.	Xây dựng hệ thống hoàn chỉnh	79
4.1.	Kiến trúc tổng thể.....	79
4.1.1.	Mô hình hệ thống	79
4.1.2.	Nguyên tắc.....	79
4.2.	System Architecture	80

4.2.1.	Overview	80
4.2.2.	Microservice Architecture	80
4.2.3.	API Gateway	80
4.2.5.	Containerization and Deployment	81
4.2.6.	System Structure	81
4.2.7.	Design Principles	82
4.2.8.	Security Considerations	83
4.2.9.	Discussion	83
4.3.	API gateway (Nginx)	84
4.3.1.	Vai trò	84
4.3.2.	Cấu hình mẫu	84
4.4.	Authentication (JWT)	85
4.4.1.	Cài đặt	86
4.4.2.	Cấu hình	87
4.4.3.	Luồng	87
4.5.	Giao tiếp giữa các Service	87
4.5.1.	REST API call	87
4.5.2.	Best Practice	88
4.6.	Docker hóa hệ thống	88
4.6.1.	Dockerfile (Django)	88
4.6.2.	Docker-compose.yml	88
4.7.	Luồng hệ thống (End-to-End)	89
4.7.1.	Use case: Mua hàng	89
4.7.2.	Sequence logic	90
4.7.3.	Giao diện	91
4.8.	Triển khai Kubernetes (Optional)	93
4.8.1.	Deployment	93
4.8.2.	Service	93
4.9.	Logging và Monitoring	94
4.10.	Đánh giá hệ thống	94
4.10.1.	Hiệu năng	94
4.10.2.	Khả năng mở rộng	94
4.10.3.	Ưu điểm	95

4.10.4. Nhược điểm	95
4.11. Bài tập thực hành	95
4.12. Checklist đánh giá	96
I. Mở đầu	96
II. ĐÁNH GIÁ CHƯƠNG 1 – TỔNG QUAN KIẾN TRÚC MICROSERVICES	97
1. So sánh Monolithic và Microservices	97
2. Phân tích tư duy kiến trúc.....	98
3. Ví dụ Healthcare Microservices.....	98
III. ĐÁNH GIÁ CHƯƠNG 2 – THIẾT KẾ HỆ THỐNG E-COMMERCE	99
1. Phân tích yêu cầu hệ thống.....	99
2. Phân rã Microservices	99
3. Thiết kế Database cho từng Service	100
4. API Design	101
5. UML và mô hình hóa hệ thống	102
IV. ĐÁNH GIÁ CHƯƠNG 3 – AI SERVICE.....	103
1. Ý tưởng tích hợp AI vào E-Commerce	103
2. Kiến trúc AI Service.....	103
3. Mô hình LSTM.....	104
4. Knowledge Graph và Neo4j	104
5. RAG và Chatbot.....	105
V. ĐÁNH GIÁ CHƯƠNG 4 – TRIỂN KHAI HỆ THỐNG	105
1. Công nghệ sử dụng.....	105
2. Triển khai và vận hành.....	106
3. Hạn chế của phần triển khai	106
VI. ĐÁNH GIÁ TỔNG HỢP	107
VII. NHỮNG ĐIỂM CẦN CẢI THIỆN	108
VIII. KẾT LUẬN.....	108
SOURCE CODE:	
https://github.com/lamlocan5/Software_Architecture_And_Design_PTIT	109

DANH MỤC HÌNH ẢNH

Hình 1. So sánh Monolithic và Microservices	14
Hình 3. Cấu trúc thư mục healthcare – 1	20
Hình 4. Cấu trúc thư mục Healthcare – 2.....	21
Hình 5. Cart Service.....	28
Hình 6. Sequence Diagram – Logic thêm sản phẩm.....	29
Hình 7. Sequence Diagram Logic xem giỏ hàng	30
Hình 8. Workflow	34
Hình 9. Sequence Diagram - Logic tạo đơn hàng	35
Hình 10. Workflow	39
Hình 11. Workflow	43
Hình 12	53
Hình 13	55
Hình 14	75
Hình 15	79
Hình 16	82
Hình 17	83
Hình 18	86
Hình 19	90

DANH MỤC BẢNG BIỂU

Bảng 1. So sánh Monolithic và Microservice	15
Bảng 2. Bảng phân quyền RBAC	27
Bảng 3. Bảng Order Service	31
Bảng 4. Bảng trạng thái	32
Bảng 5. Bảng orderItem	33
Bảng 6. Bảng Payment.....	37
Bảng 7. Các method.....	37
Bảng 8. Các trạng thái.....	38
Bảng 9. Bảng API	40
Bảng 10. Bảng Shipping Service	42
Bảng 11. Bảng trạng thái	43
Bảng 12. Bảng API	44
Bảng 13. Kiến trúc tổng quan	45
Bảng 14. Phân loại vai trò người dùng	46

1. Từ Monolithic đến Microservices và DDD

Trong lịch sử phát triển phần mềm, sự thay đổi kiến trúc thường không đến từ ý thích cá nhân mà đến từ nhu cầu cấp bách của thị trường: tốc độ (Velocity) và khả năng mở rộng (Scalability). Khi một hệ thống phần mềm bắt đầu vượt quá khả năng quản lý của một nhóm phát triển đơn lẻ, các vấn đề về hiệu suất và tính linh hoạt sẽ nảy sinh.

Báo cáo này phân tích lộ trình đi từ "khối thống nhất" (Monolithic) sang "hệ sinh thái dịch vụ" (Microservices) dưới sự định hướng của "Thiết kế hướng tên miền" (Domain Driven Design - DDD) - một kỹ thuật giúp phân rã hệ thống dựa trên nghiệp vụ thay vì dựa trên hạ tầng kỹ thuật.

1.1. Giới thiệu Monolithic Architecture

1.1.1. Khái niệm

Monolithic Architecture là một mô hình kiến trúc phần mềm trong đó toàn bộ mã nguồn của các thành phần chức năng (UI, Business Logic, Data Access) được biên dịch và đóng gói thành một đơn vị triển khai duy nhất (ví dụ: một file .war, .ear, hoặc một file thực thi duy nhất). Trong kiến trúc này, các module giao tiếp với nhau thông qua việc gọi hàm trực tiếp trong cùng một không gian bộ nhớ (In-memory calls).

1.1.2. Cấu trúc điển hình

Một ứng dụng Monolithic thường tuân theo mô hình phân lớp (Layered Architecture):

- **Presentation Layer (Lớp hiển thị):** Xử lý giao diện người dùng, tiếp nhận yêu cầu HTTP và trả về phản hồi.
- **Business Logic Layer (Lớp nghiệp vụ):** Nơi chứa toàn bộ linh hồn của ứng dụng, các quy tắc kinh doanh, xử lý tính toán.

- **Data Access Layer (Lớp truy cập dữ liệu):** Thực hiện các truy vấn CRUD xuống cơ sở dữ liệu.
- **Database (Cơ sở dữ liệu):** Thường là một RDBMS duy nhất (MySQL, PostgreSQL, SQL Server) lưu trữ toàn bộ schema của hệ thống.

1.1.3. Ví dụ thực tế

Hãy tưởng tượng một ứng dụng Thương mại điện tử giai đoạn đầu. Module quản lý người dùng, module sản phẩm, module giỏ hàng và module thanh toán đều nằm trong cùng một project Java hoặc Node.js. Khi bạn muốn cập nhật màu sắc của nút "Mua ngay", bạn phải build lại toàn bộ project và khởi động lại server.

1.1.4. Nhược điểm chi tiết

- Quá trình CI/CD chậm chạp: Khi codebase lên tới hàng triệu dòng code, việc chạy Unit Test và biên dịch có thể mất hàng giờ. Điều này ngăn cản việc triển khai liên tục.
- Sự phụ thuộc lẫn nhau (Tight Coupling): Một thay đổi nhỏ ở module Thanh toán có thể vô tình làm hỏng module Kho hàng do các hàm gọi chéo nhau không kiểm soát.
- Rào cản công nghệ (Technology Lock-in): Nếu bạn bắt đầu với Java 8, việc nâng cấp lên Java 17 cho toàn bộ hệ thống là một rủi ro cực lớn. Bạn không thể sử dụng Python cho một tính năng AI nhỏ trong khi toàn bộ app là Java.
- Khả năng mở rộng kém hiệu quả: Để đáp ứng lượng truy cập tăng đột biến vào module tìm kiếm, bạn buộc phải nhân bản toàn bộ ứng dụng (bao gồm cả các module đang rảnh rỗi như thanh toán), gây lãng phí tài nguyên RAM/CPU.
- Độ phức tạp nhận thức: Lập trình viên mới mất rất nhiều thời gian để hiểu được luồng dữ liệu trong một "Big Ball of Mud" (Quả cầu bùn khổng lồ).

1.1.5. Khi nào nên dùng Monolithic

Dù có nhiều nhược điểm, Monolithic vẫn là lựa chọn tối ưu cho:

- **MVP (Minimum Viable Product):** Khi bạn cần đưa sản phẩm ra thị trường nhanh nhất để kiểm chứng ý tưởng.
- **Nhóm nhỏ (dưới 5-7 người):** Việc quản lý Microservices sẽ tốn nhiều thời gian hơn là viết code tính năng.
- **Ứng dụng đơn giản:** Nếu nghiệp vụ không quá phức tạp, Monolithic giúp giảm thiểu chi phí vận hành hạ tầng (Network, Monitoring).

1.2. Microservices Architecture

1.2.1. Khái niệm

Microservices là kiến trúc phần mềm trong đó hệ thống được chia thành nhiều dịch vụ nhỏ độc lập. Mỗi service thực hiện một chức năng riêng biệt và có thể được phát triển, triển khai, bảo trì độc lập với các service khác.

Các service giao tiếp với nhau thông qua API hoặc message broker như REST API, gRPC hoặc Kafka.

Ví dụ trong hệ thống thương mại điện tử có thể bao gồm:

- Service quản lý người dùng
- Service thanh toán
- Service giỏ hàng
- Service tìm kiếm sản phẩm
- Service gửi thông báo

Mỗi service có cơ sở dữ liệu riêng và có thể sử dụng công nghệ khác nhau tùy theo yêu cầu.

Lý do các BigTech sử dụng Microservices

- Các công ty công nghệ lớn thường có:

- Hàng triệu người dùng đồng thời
- Khối lượng dữ liệu khổng lồ
- Nhiều nhóm phát triển cùng làm việc
- Yêu cầu uptime và hiệu năng cao

Microservices giúp các công ty:

- Dễ mở rộng hệ thống
- Tăng tốc phát triển tính năng
- Giảm ảnh hưởng khi xảy ra lỗi
- Hỗ trợ triển khai liên tục (CI/CD)
- Tăng khả năng chịu tải

a) Quá trình phát triển Microservices tại Netflix

Giai đoạn đầu

Netflix ban đầu sử dụng kiến trúc Monolithic truyền thống. Khi số lượng người dùng tăng nhanh, hệ thống thường xuyên gặp tình trạng quá tải và downtime.

Chuyển sang Microservices

Khoảng giai đoạn 2009–2012, Netflix bắt đầu chuyển đổi sang kiến trúc Microservices trên nền tảng cloud của Amazon Web Services.

Hệ thống được chia thành hàng trăm service nhỏ như:

- Recommendation service
- Streaming service
- User service
- Billing service

Công nghệ nổi bật

Netflix phát triển nhiều công cụ nổi tiếng:

- Eureka – Service Discovery
- Hystrix – Fault Tolerance
- Zuul – API Gateway

Kết quả

- Có thể deploy hàng nghìn lần mỗi ngày
- Hệ thống ổn định hơn
- Tăng khả năng mở rộng toàn cầu

b) Amazon

Giai đoạn đầu

Amazon từng sử dụng hệ thống nguyên khối rất lớn. Khi quy mô tăng nhanh, việc phát triển trở nên khó khăn vì các module phụ thuộc lẫn nhau.

Mô hình “Two Pizza Team”

Jeff Bezos đề xuất mô hình:

Một nhóm nên nhỏ đến mức hai chiếc pizza đủ cho cả nhóm ăn.

Mỗi team quản lý một service độc lập.

API-first

Amazon yêu cầu mọi service phải giao tiếp thông qua API nội bộ. Điều này tạo nền tảng cho:

- SOA
- Microservices
- AWS sau này

Kết quả

- Amazon đạt được khả năng:
- Scale cực lớn
- Phát triển song song
- Triển khai nhanh chóng

c) Google

Google từ sớm đã xây dựng hệ thống phân tán quy mô lớn.

Công nghệ ảnh hưởng đến Microservices hiện đại

Google phát triển nhiều công nghệ nổi tiếng:

- Kubernetes
- gRPC
- Borg
- Istio

Định hướng

- Google tập trung vào:
- Container orchestration
- Scalability
- Reliability
- Observability

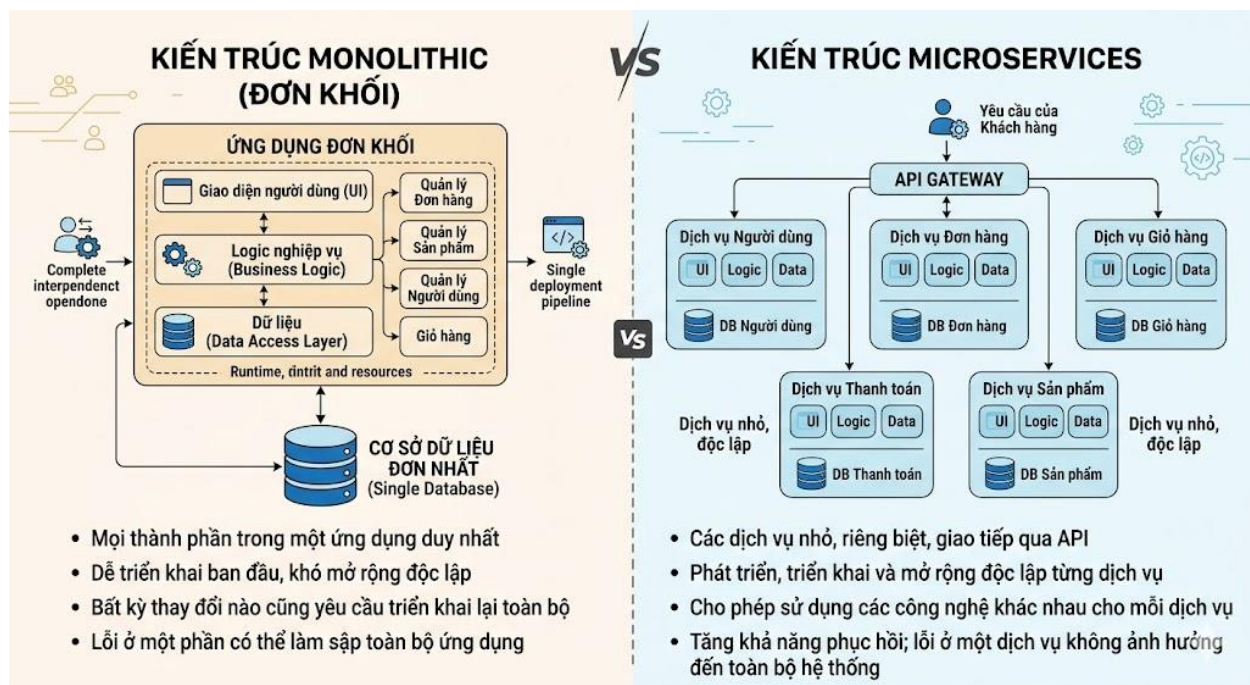
Những công nghệ này hiện được sử dụng rộng rãi trong hệ sinh thái Cloud Native.

1.2.2. Đặc điểm

- Decentralization (Phi tập trung): Không có sự quản trị tập trung về dữ liệu hay công nghệ. Mỗi service chọn công nghệ phù hợp nhất.

- Componentization via Services: Các thành phần được module hóa thông qua các dịch vụ thay vì thư viện (library).
- Business Capability: Service được tổ chức quanh các năng lực nghiệp vụ (ví dụ: Service Giao hàng, Service Khuyến mãi).
- Evolutionary Design: Kiến trúc có thể tiến hóa dần dần mà không cần đập đi xây lại toàn bộ.

1.2.3. So sánh Monolithic và Microservices



Hình 1. So sánh Monolithic và Microservices

Tiêu chí	Monolithic	Microservices
Độ phức tạp phát triển	Thấp ở giai đoạn đầu	Cao ngay từ đầu

Quản lý dữ liệu	Tập trung (Centralized DB)	Phân tán (Database per Service)
Tính nhất quán	Strong Consistency (ACID)	Eventual Consistency (BASE)
Giao tiếp	Gọi hàm trực tiếp (Fast)	Qua mạng (Latency)
Khả năng chịu lỗi	Một lỗi nhỏ có thể sập toàn bộ	Lỗi được cô lập trong service

Bảng 1. So sánh Monolithic và Microservice

1.2.4. Ưu điểm

- Khả năng Scale độc lập: Chỉ tăng tài nguyên cho service bị nghẽn.
- Đội ngũ tự chủ: Nhóm làm service A không cần chờ nhóm làm service B xong mới deploy được.
- Thử nghiệm công nghệ mới: Có thể viết service mới bằng Go hoặc Rust trong khi hệ thống cũ vẫn chạy Java.

1.2.5. Nhược điểm

- Độ phức tạp hạ tầng: Cần hệ thống Logging, Tracing (Jaeger, Zipkin) và Monitoring (Prometheus, Grafana) cực tốt.
- Vấn đề mạng: Network không bao giờ tin cậy hoàn toàn. Cần xử lý Retry, Circuit Breaker.
- Giao dịch phân tán: Việc đảm bảo dữ liệu khớp nhau giữa 10 database khác nhau là một bài toán hóc búa (Saga Pattern).

1.2.6. Nguyên tắc thiết kế

- Loose Coupling & High Cohesion: Các dịch vụ ít phụ thuộc nhau nhưng bên trong mỗi dịch vụ các thành phần phải liên kết chặt chẽ.
- Automation: Tự động hóa mọi thứ từ Test đến Deploy.
- Hide Internal Implementation: Chỉ lộ ra những gì cần thiết qua API.

1.3. Domain Driven Design (DDD)

1.3.1. Mục tiêu

DDD sinh ra để giải quyết vấn đề: Làm sao để chia nhỏ một cục Monolithic khổng lồ thành các Microservices mà không bị "sai" về mặt nghiệp vụ? DDD tập trung vào việc hiểu sâu sắc mô hình kinh doanh thay vì chỉ tập trung vào code.

1.3.2. Các khái niệm cốt lõi

- Ubiquitous Language (Ngôn ngữ chung): Đội ngũ Dev và đội ngũ Business phải dùng chung một bộ thuật ngữ. Nếu Business gọi là "Khách hàng thân thiết", Dev không được đặt tên biến là VipUser.
- Bounded Context (Ngữ cảnh giới hạn): Một ranh giới logic mà trong đó các mô hình và ngôn ngữ có ý nghĩa nhất định. Ví dụ: Từ "Sản phẩm" trong Context Bán hàng có giá cả, nhưng trong Context Kho hàng nó lại có kích thước và cân nặng.
- Entity & Value Object: Entity có định danh (ID), Value Object chỉ là các thuộc tính (ví dụ: Địa chỉ).
- Aggregate Root: "Người gác cổng" cho một nhóm các object liên quan, đảm bảo tính toàn vẹn dữ liệu.

1.3.3. Context Map

Đây là công cụ đồ họa thể hiện cách các Bounded Context tương tác với nhau (Upstream/Downstream, Customer/Supplier). Nó giúp kiến trúc sư thấy được bức tranh tổng thể về dòng chảy dữ liệu.

1.3.4. DDD trong Microservices

DDD cung cấp ranh giới để xác định kích cỡ của một Microservice. Thông thường, một Bounded Context sẽ tương ứng với một Microservice. Điều này đảm bảo mỗi service có một "trách nhiệm" rõ ràng và không bị lẫn lộn sang nhau.

1.4. Case Study: Healthcare (Luyện Decomposition)

1.4.1. Mô tả bài toán

Một bệnh viện đang chạy hệ thống cũ quản lý mọi thứ từ lịch hẹn, hồ sơ bệnh án đến viện phí trong một app duy nhất. Hệ thống thường xuyên bị treo vào giờ cao điểm khi bệnh nhân đăng ký khám đông.

1.4.2. Bước 1: Xác định Domain

- Core Domain: Khám chữa bệnh (Clinical).
- Supporting Domain: Quản lý lịch hẹn (Scheduling).
- Generic Domain: Thanh toán (Payment), Thông báo (Notification).

1.4.3. Bước 2: Xác định Bounded Context

Sau khi phỏng vấn bác sĩ và nhân viên y tế, ta xác định được các vùng độc lập:

- Patient Context: Quản lý hồ sơ định danh bệnh nhân.
- Medical Record Context: Quản lý kết quả xét nghiệm, lịch sử bệnh lý.
- Finance Context: Quản lý bảo hiểm, hóa đơn, thanh toán.
- Pharmacy Context: Quản lý thuốc và vật tư y tế.

1.4.4. Bước 3: Phân rã thành Microservices

Từ các Context trên, ta thiết kế các dịch vụ:

- Patient-Service: Lưu trữ thông tin cá nhân.
- Clinical-Service: Xử lý logic khám bệnh.
- Billing-Service: Xử lý tiền nong.
- Inventory-Service: Quản lý kho thuốc.

1.4.5. Bước 4: Xác định quan hệ

- Khi bác sĩ kê đơn (trong Clinical-Service), một sự kiện PrescriptionCreated được bắn ra.
- Inventory-Service nghe sự kiện này để trừ kho thuốc.

- Billing-Service nghe sự kiện này để tạo hóa đơn tạm tính.
- ⇒ Đây là kiểu giao tiếp bất đồng bộ (Event-driven) giúp hệ thống cực kỳ linh hoạt.

1.4.6. Ví dụ API

- POST /api/v1/appointments: Tạo lịch hẹn (Appointment Service).
- GET /api/v1/patients/{id}/records: Xem bệnh án (Medical Record Service - có thể gọi sang Patient Service để lấy tên).
- PUT /api/v1/bills/{id}/pay: Thanh toán hóa đơn (Finance Service).

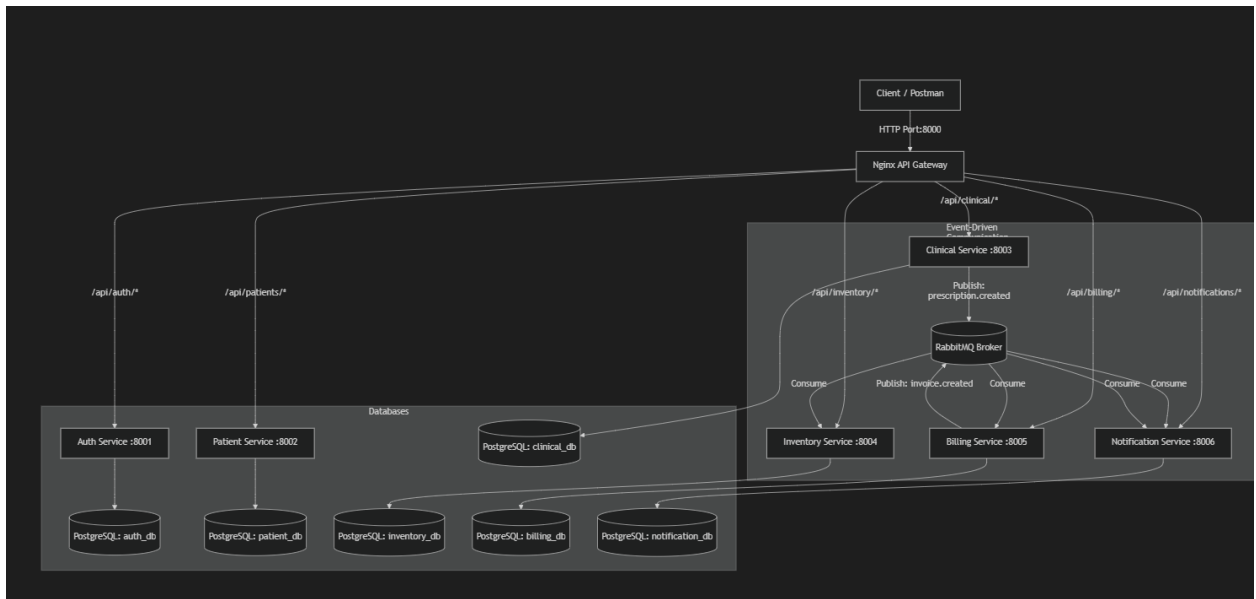
1.5. Kết luận

Chuyển đổi từ Monolithic sang Microservices không phải là một đích đến mà là một hành trình liên tục tối ưu.

- Về mặt kỹ thuật: Chúng ta đổi lấy sự đơn giản của code lấy sự phức tạp của hệ thống mạng.
- Về mặt con người: Chúng ta cần một văn hóa DevOps mạnh mẽ, nơi lập trình viên có trách nhiệm với cả việc vận hành sản phẩm mình làm ra ("You build it, you run it").

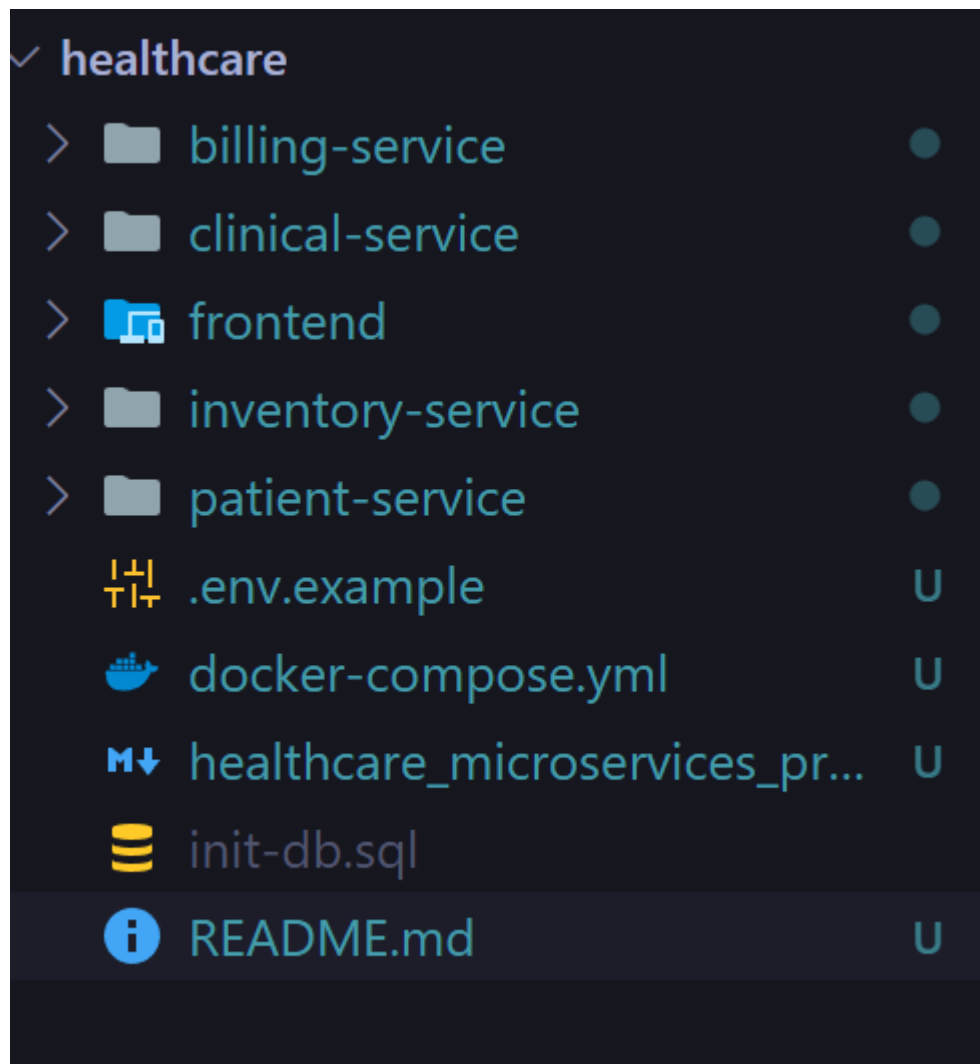
Ứng dụng DDD là cách duy nhất để đảm bảo rằng kiến trúc Microservices của bạn không trở thành một đống hỗn độn các dịch vụ gọi nhau chằng chịt. Hãy bắt đầu từ nhỏ (Monolith), hiểu rõ Domain, và chỉ phân rã khi thực sự cần thiết.

- Kiến trúc hệ thống:

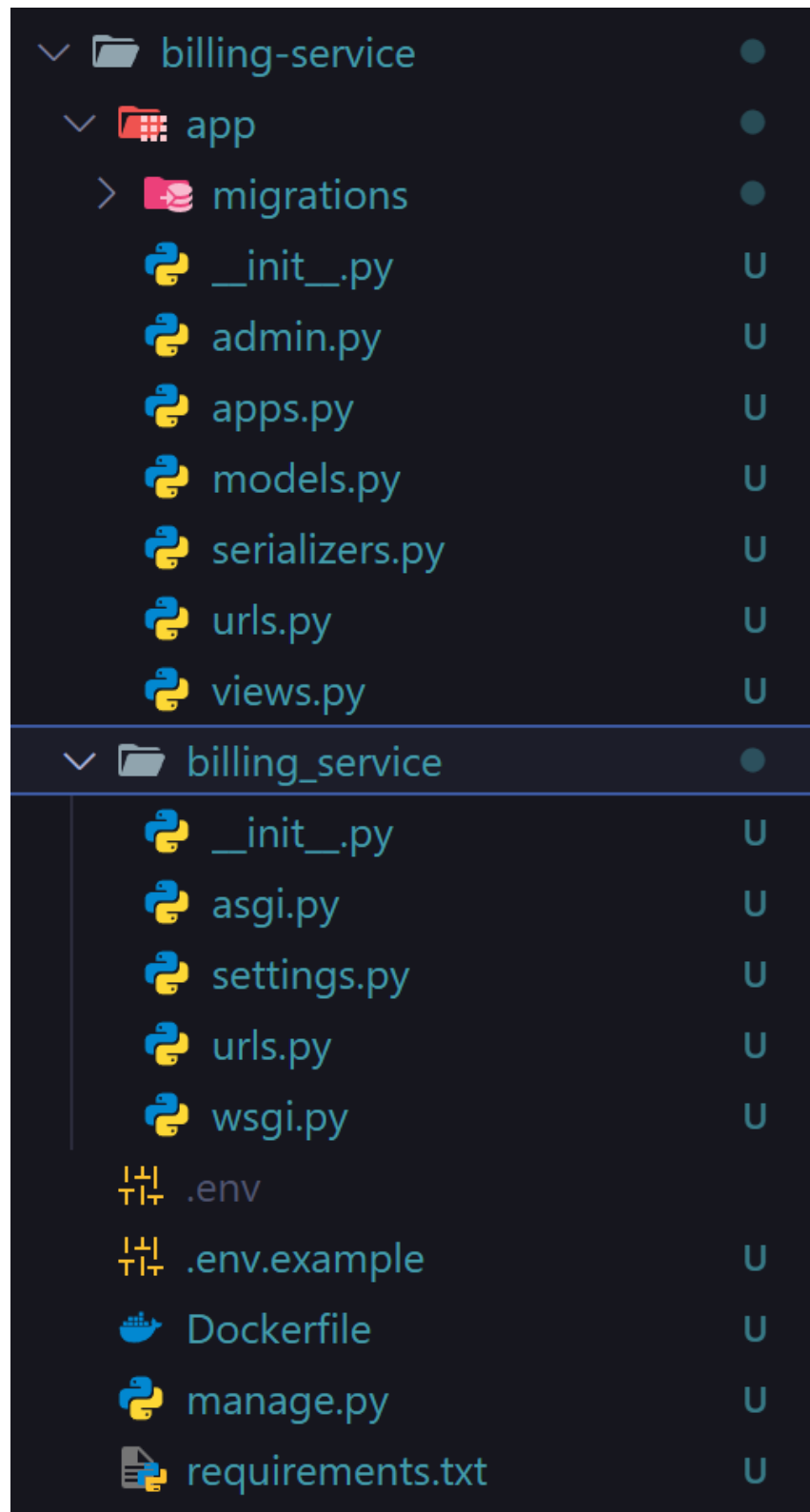


Hình 2

Ảnh cấu trúc thư mục code



Hình 3. Cấu trúc thư mục healthcare – 1

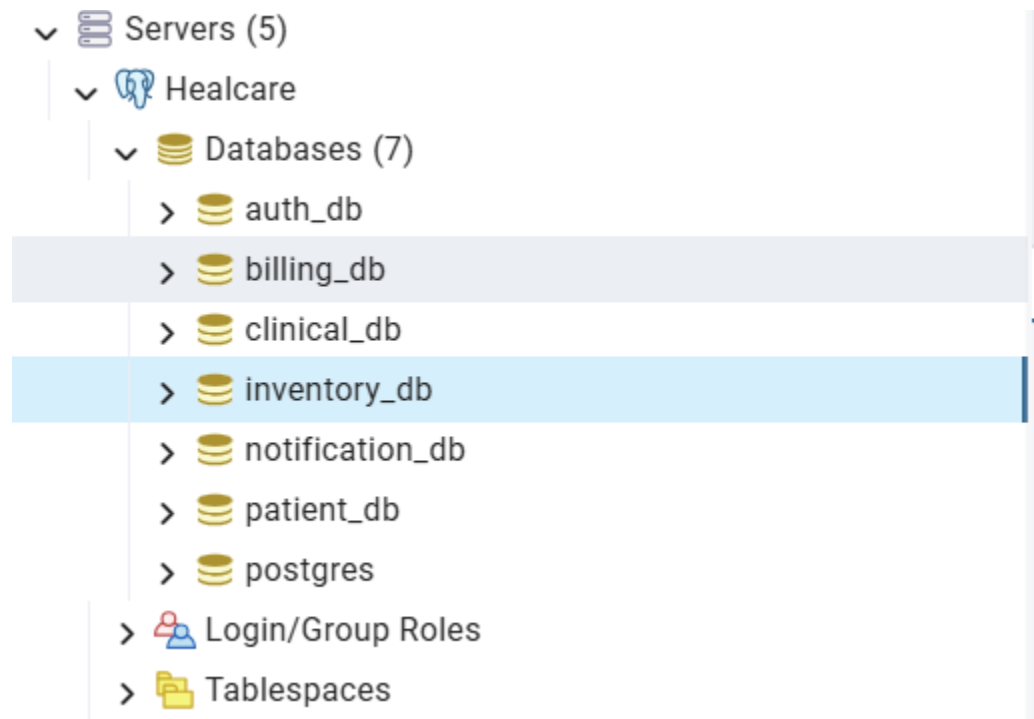


Hình 4. Cấu trúc thư mục Healthcare – 2

☐	▼	●	healcare	-	-	-	6.84%	886.48MB / 126.1	7.51%	40.2MB	🔗	■	⋮	🗑
☐		●	hospital_gate	522392a340f1	nginx:alpin	8000:8000	0%	2.65MB / 11.52G	0.02%	0B / 12.	🔗	■	⋮	🗑
☐		●	clinical_servi	35742a3e6f06	healcare-cl		0.91%	93.77MB / 11.52	0.79%	0B / 0B	🔗	■	⋮	🗑
☐		●	billing_servic	529c2df8d935	healcare-bi		1.22%	91.52MB / 11.52	0.78%	0B / 0B	🔗	■	⋮	🗑
☐		●	inventory_cor	34d93debf92f	healcare-in		0%	48.47MB / 11.52	0.41%	0B / 0B	🔗	■	⋮	🗑
☐		●	notification_s	18f696999237	healcare-ni		1.12%	91.28MB / 11.52	0.77%	0B / 0B	🔗	■	⋮	🗑
☐		●	patient_servi	0e1f9c916f56	healcare-pi		1.01%	91.54MB / 11.52	0.78%	0B / 0B	🔗	■	⋮	🗑
☐		●	auth_service	14ba5564d435	healcare-ai		1.1%	91.55MB / 11.52	0.78%	0B / 0B	🔗	■	⋮	🗑
☐		●	billing_cons	2d9f4ad73ed0	healcare-bi		0%	48.2MB / 11.52G	0.41%	0B / 0B	🔗	■	⋮	🗑
☐		●	notification_s	18f696999237	healcare-ni		0.93%	91.28MB / 11.52	0.77%	0B / 0B	🔗	■	⋮	🗑
☐		●	patient_servi	0e1f9c916f56	healcare-pi		0.99%	91.54MB / 11.52	0.78%	0B / 0B	🔗	■	⋮	🗑
☐		●	auth_service	14ba5564d435	healcare-ai		1.11%	91.55MB / 11.52	0.78%	0B / 0B	🔗	■	⋮	🗑
☐		●	billing_cons	2d9f4ad73ed0	healcare-bi		0%	48.2MB / 11.52G	0.41%	0B / 0B	🔗	■	⋮	🗑
☐		●	inventory_ser	6589e3fed1c8	healcare-in		1.02%	92.3MB / 11.52G	0.78%	0B / 0B	🔗	■	⋮	🗑
☐		●	hospital_db	4d4ab66e3f59	postgres:1	5432:5432	0.11%	89.03MB / 11.52	0.75%	40.2MB	🔗	■	⋮	🗑
☐		●	hospital_rab	851fc1c5514f	rabbitmq:3	15672:15672 Show all ports (2)	0.2%	146.2MB / 11.52	1.24%	0B / 70	🔗	■	⋮	🗑

Hình 5

- Database:



Hình 6

2. Phát triển Hệ E-Commerce Microservices

Sau khi đã nắm vững nền tảng về Microservices và Domain-Driven Design (DDD) ở chương trước, chương này sẽ tập trung vào việc hiện thực hóa một hệ thống Thương mại điện tử (E-Commerce) hoàn chỉnh. Chúng ta sẽ đi từ khâu xác định yêu cầu đến thiết kế chi tiết cho từng dịch vụ thành phần.

2.1. Xác định yêu cầu

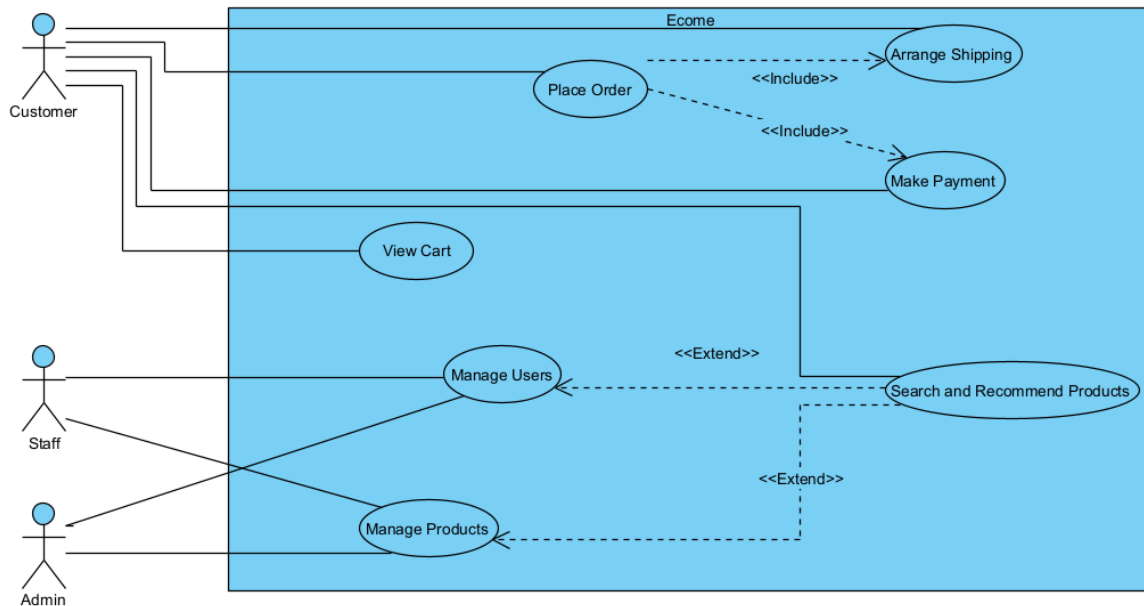
Để xây dựng một hệ thống E-commerce bền vững, việc xác định rõ ràng các yêu cầu là bước tiên quyết.

2.1.1. Functional Requirements

Các chức năng cốt lõi mà người dùng và quản trị viên có thể thực hiện:

- Quản lý sản phẩm: Cho phép người dùng tìm kiếm, xem chi tiết sản phẩm, phân loại theo danh mục.
- Quản lý người dùng: Đăng ký, đăng nhập, quản lý thông tin cá nhân và địa chỉ giao hàng.
- Giỏ hàng: Thêm/bớt sản phẩm, cập nhật số lượng và lưu trữ trạng thái giỏ hàng tạm thời.
- Đặt hàng & Thanh toán: Quy trình tạo đơn hàng, tính toán tổng tiền, tích hợp các cổng thanh toán.
- Vận chuyển: Theo dõi trạng thái giao hàng, thông tin đơn vị vận chuyển.

Usecase Tổng quan:



Hình 7. Usecase Tổng quan

2.1.2. Non-functional Requirements

Các tiêu chuẩn về chất lượng vận hành:

- Tính sẵn sàng (Availability): Hệ thống phải hoạt động 24/7, ngay cả khi một dịch vụ gặp sự cố, các phần khác vẫn phải hoạt động.
- Khả năng mở rộng (Scalability): Có khả năng chịu tải lớn trong các dịp khuyến mãi (Flash Sale).
- Bảo mật: Mã hóa thông tin thanh toán, bảo vệ dữ liệu người dùng (GDPR/khung bảo mật tương đương).
- Hiệu suất: Thời gian phản hồi API phải dưới 200ms cho các tác vụ thông thường.

2.2. Phân rã hệ thống theo DDD

Việc phân rã không dựa trên kỹ thuật mà dựa trên nghiệp vụ thực tế.

2.2.1. Bounded Context

Chúng ta xác định 6 Bounded Context chính, mỗi vùng sẽ tương ứng với một Microservice độc lập:

- Product Context: Chịu trách nhiệm về danh mục và thông tin hàng hóa.
- User Context: Chịu trách nhiệm về định danh và phân quyền.
- Cart Context: Quản lý trạng thái mua sắm tạm thời.
- Order Context: Quản lý quy trình bán hàng và lịch sử đơn hàng.
- Payment Context: Xử lý giao dịch tài chính.
- Shipping Context: Xử lý logistics và giao nhận.

2.2.2. Nguyên tắc

- Loose Coupling: Các service không được truy cập trực tiếp vào Database của nhau.
- High Cohesion: Các logic liên quan chặt chẽ phải nằm trong cùng một service.
- Single Source of Truth: Mỗi mẫu dữ liệu (ví dụ: Giá sản phẩm) phải có một service "chủ quản" duy nhất.

2.3. Thiết kế Product Service (Django)

2.3.1. Phân loại sản phẩm

Sản phẩm được tổ chức theo cấu trúc cây (Category Tree) và các thuộc tính động (Attributes) như màu sắc, kích thước để phục vụ bộ lọc tìm kiếm.

2.3.2. Model tổng quát

Sử dụng Django Models để định nghĩa cấu trúc dữ liệu:

- Category: id, name, slug, parent_id.
- Product: id, name, description, base_price, category_id, created_at.

2.3.3. Chi tiết theo domain

Để xử lý các biến thể sản phẩm (Ví dụ: iPhone 15 Pro - Đen - 256GB), chúng ta cần:

- ProductVariant: id, product_id, sku, price_override, stock_quantity.
- Attribute: id, name (e.g., Color).
- AttributeValue: id, attribute_id, value (e.g., Black).

2.3.4. API

- GET /products/: Danh sách sản phẩm kèm bộ lọc.
- GET /products/{slug}/: Chi tiết sản phẩm và các biến thể.
- POST /admin/products/: Thêm mới sản phẩm (Dành cho Admin).

2.4. Thiết kế User Service (Django)

2.4.1. Phân loại người dùng

- Guest: Chỉ xem sản phẩm.
- Customer: Mua hàng, quản lý hồ sơ.
- Admin/Staff: Quản trị hệ thống và xử lý đơn hàng.

2.4.2. Model

Mở rộng AbstractUser của Django:

- User: email (username), full_name, phone_number, is_active.
- Address: user_id, street, city, country, is_default.

2.4.3. Phân quyền (RBAC)

Sử dụng nhóm (Groups) và quyền (Permissions) để kiểm soát truy cập API. Ví dụ: Chỉ người có quyền can_edit_product mới được gọi API cập nhật giá.

Quyền hạn (Permission)	Guest	Customer	Staff	Admin
Xem sản phẩm	X	X	X	X
Đặt hàng		X	X	X
Quản lý kho hàng			X	X

Hủy đơn hàng của khách			X	X
Xóa tài khoản người dùng				X
Cấu hình hệ thống				X

Bảng 2. Bảng phân quyền RBAC

2.4.4. API

- POST /auth/register/: Đăng ký tài khoản
- POST /auth/login/: Trả về JWT (JSON Web Token).
- GET /users/me/: Lấy thông tin cá nhân hiện tại.

2.5. Thiết kế Cart Service (Django)

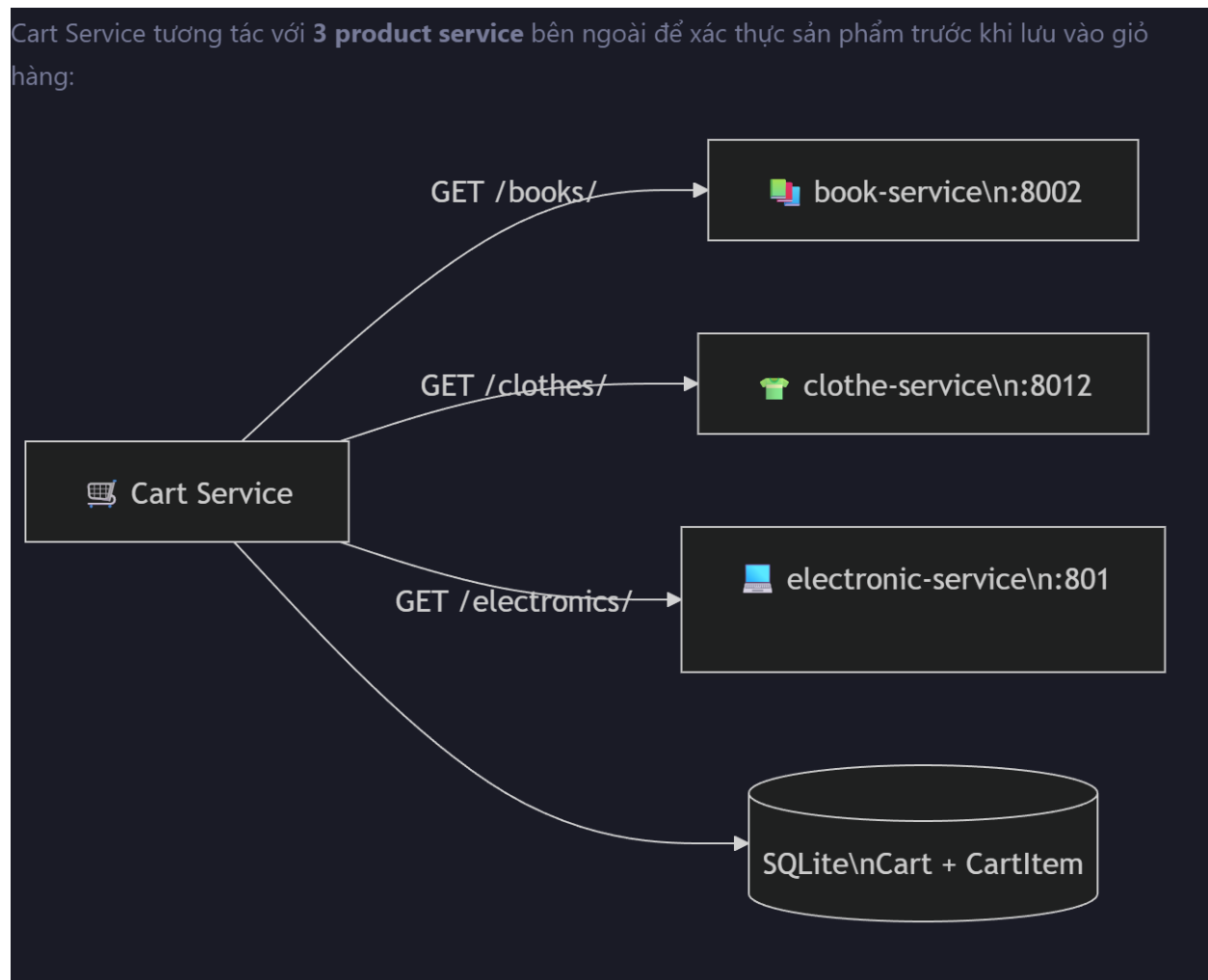
2.5.1. Model

Cấu trúc dữ liệu bao gồm hai bảng chính với mối quan hệ 1-n (Một-Nhiều): Một giỏ hàng (Cart) có thể chứa nhiều mục sản phẩm (CartItem).

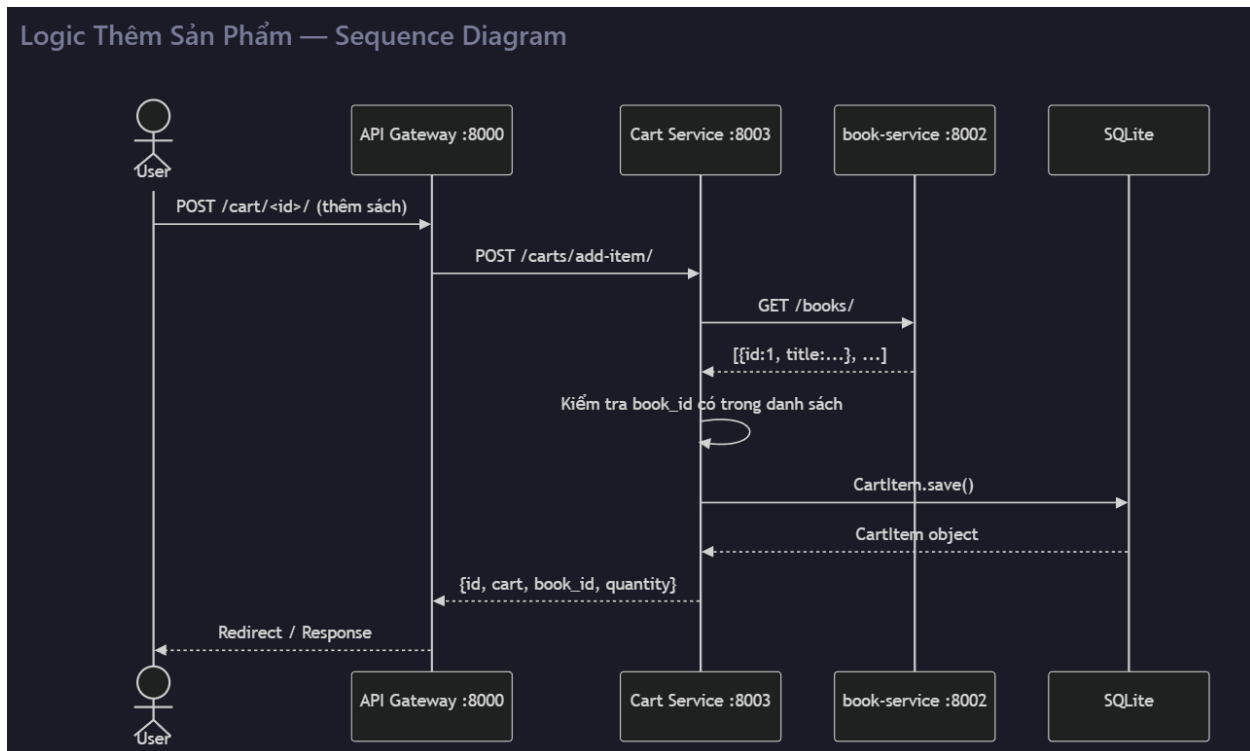
- Model Cart (Giỏ hàng):
 - Vai trò: Đại diện cho một phiên mua sắm của một khách hàng cụ thể.
 - Trường dữ liệu:
 - id: Khóa chính định danh giỏ hàng.
 - customer_id: Đây là một trường Soft-link (Liên kết mềm). Thay vì sử dụng Foreign Key cứng, ta chỉ lưu ID kiểu số để tham chiếu sang Customer Service. Điều này giúp Cart Service hoạt động độc lập ngay cả khi User Service gặp sự cố.
- Model CartItem (Sản phẩm trong giỏ):
 - Vai trò: Lưu trữ chi tiết từng loại mặt hàng và số lượng.

- Thiết kế Đa Sản Phẩm (Polymorphic-like ID): Thay vì tạo 3 bảng khác nhau, chúng ta sử dụng cơ chế lưu trữ 3 trường ID tùy chọn: `book_id`, `clothing_id`, và `electronic_id`.
- Quy tắc: Tại một thời điểm, chỉ một trong ba trường này có giá trị thực (lớn hơn 0), hai trường còn lại sẽ mang giá trị null hoặc 0 (giá trị lính canh - sentinel value).

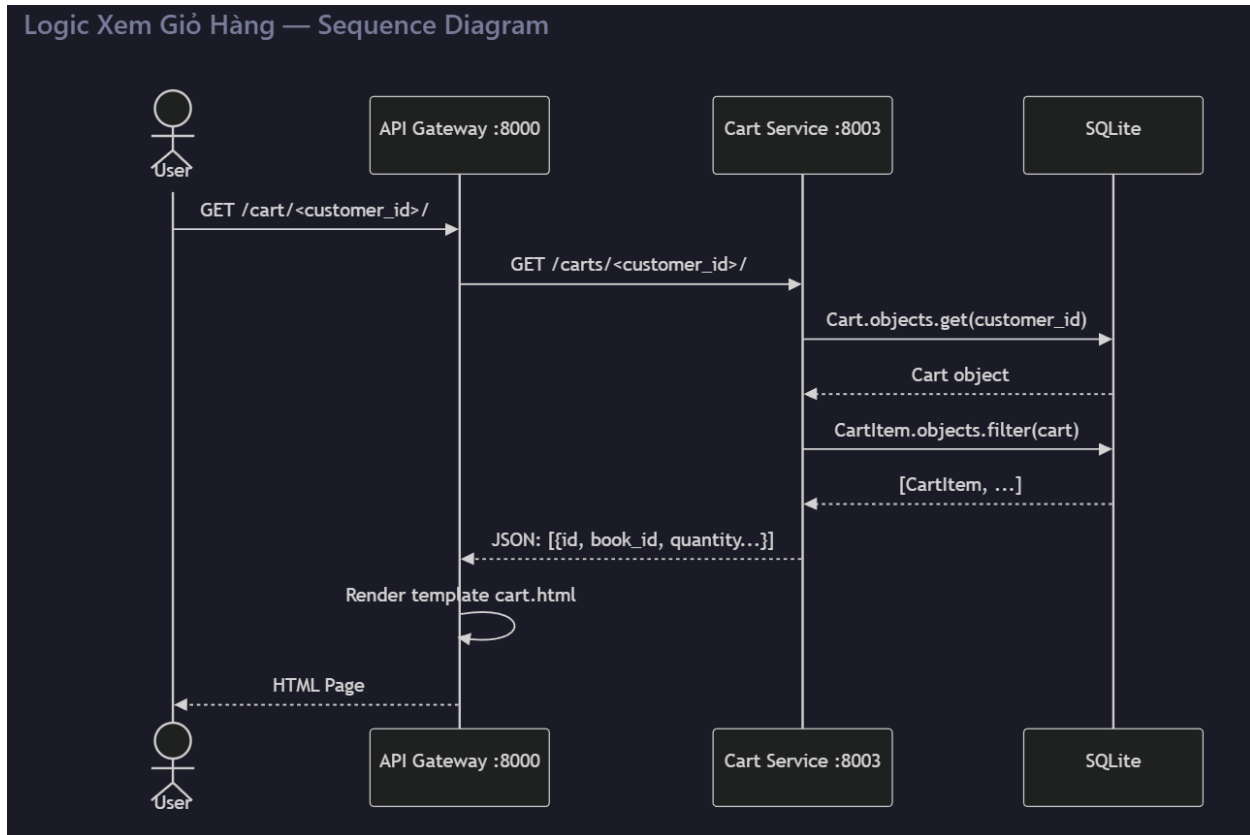
2.5.2. Logic



Hình 8. Cart Service



Hình 9. Sequence Diagram – Logic thêm sản phẩm



Hình 10. Sequence Diagram Logic xem giỏ hàng

2.5.3. API

- POST /carts/ (CartCreate):
 - Mục đích: Khởi tạo giỏ hàng ngay khi người dùng đăng ký hoặc bắt đầu phiên mua sắm.
 - Input: { "customer_id": int }.
- POST /carts/add-item/ (AddCartItem):
 - Mục đích: Thêm mới hoặc cập nhật số lượng sản phẩm.
 - Xử lý lỗi: Trả về 400 Bad Request nếu gọi tới các Product Services mà nhận được phản hồi không tìm thấy sản phẩm.
- GET /carts/<customer_id>/ (ViewCart):
 - Mục đích: Lấy toàn bộ danh sách item để hiển thị cho khách hàng.
 - Output: Một mảng các đối tượng chứa thông tin ID sản phẩm và số lượng.

- GET /carts/customer/<customer_id>/ (CartByCustomer):
- Mục đích: Kiểm tra nhanh sự tồn tại của giỏ hàng cho một người dùng nhất định.

2.6. Thiết kế Order Service (Django)

2.6.1. Model

- Order

STT	Trường dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nội dung
1	id	BigAutoField	PK, Tự tăng	Mã số định danh duy nhất của đơn hàng.
2	customer_id	IntegerField	NOT NULL	ID khách hàng (Liên kết logic sang Customer Service).
3	status	CharField(20)	Choices, Default='pending'	Trạng thái hiện tại của đơn hàng trong quy trình.
4	total_amount	DecimalField(12,2)	Default=0	Tổng giá trị tiền của đơn hàng sau khi tính toán.
5	created_at	DateTimeField	auto_now_add	Ngày và giờ hệ thống tự động ghi nhận khi tạo đơn.

Bảng 3. Bảng Order Service

- Order Status

STT	Giá trị (Key)	Nhãn hiển thị (Label)	Ý nghĩa nghiệp vụ
1	pending	Chờ xác nhận	Đơn hàng mới được tạo, chờ hệ thống/nhân viên kiểm tra.
2	confirmed	Đã xác nhận	Đơn hàng hợp lệ và đã được hệ thống chấp nhận.
3	shipping	Đang giao	Hàng đã được đóng gói và bàn giao cho đơn vị vận chuyển.
4	delivered	Đã giao	Khách hàng đã nhận hàng thành công.
5	cancelled	Đã hủy	Đơn hàng bị hủy bởi khách hàng hoặc hệ thống.

Bảng 4. Bảng trạng thái

- OrderItem

STT	Trường dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nội dung
1	id	BigAutoField	PK, Tự tăng	Mã số định danh duy nhất cho từng dòng sản phẩm.

2	order	ForeignKey Order	→ NOT NULL, CASCADE	Liên kết đến mã đơn hàng chính (Xóa đơn sẽ xóa chi tiết).
3	book_id	IntegerField	NOT NULL	ID của sách (Tham chiếu logic sang Book Service).
4	quantity	IntegerField	NOT NULL	Số lượng của sản phẩm này trong đơn hàng.
5	price_at_order	DecimalField(10,2)	NOT NULL	Giá của sản phẩm được chốt tại thời điểm đặt mua.

Bảng 5. Bảng orderItem

2.6.2. Workflow

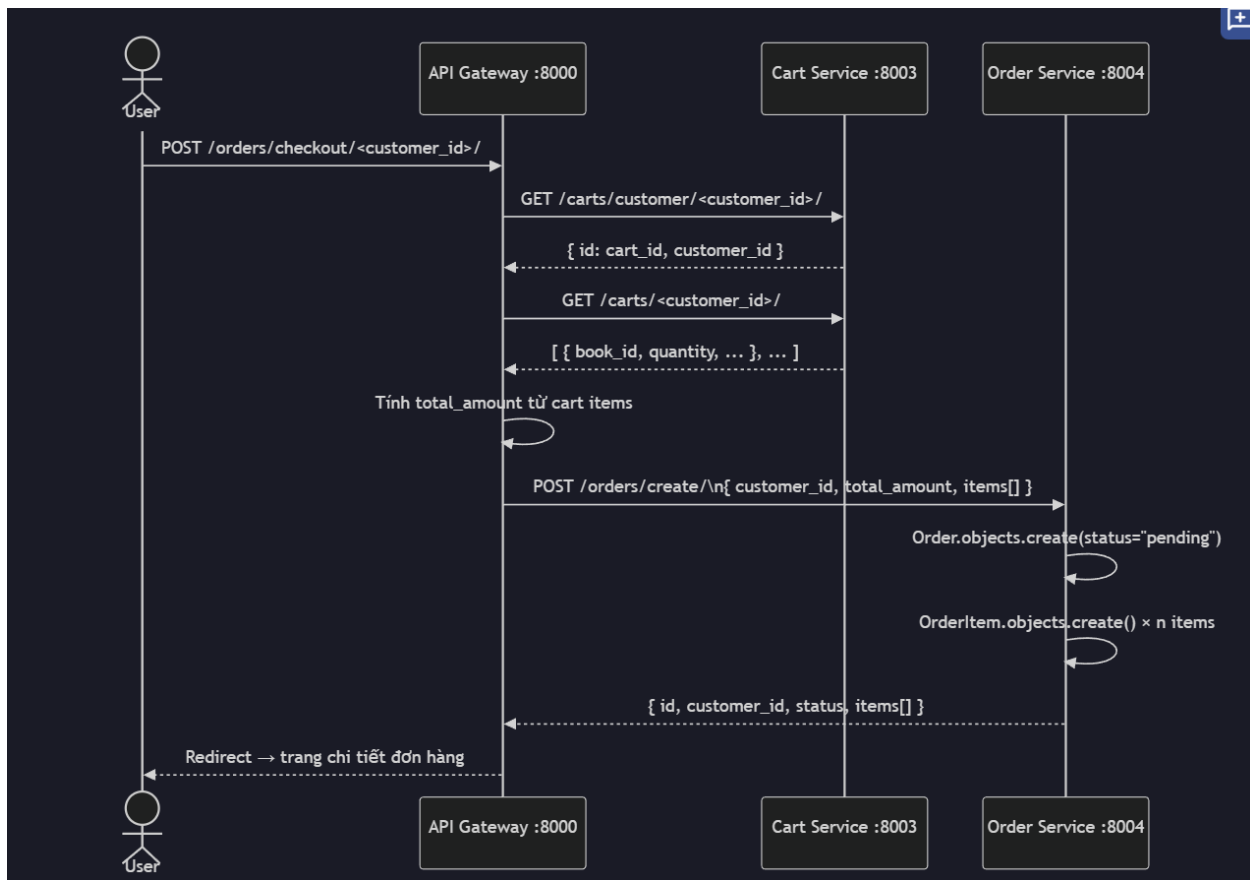
Một đơn hàng trải qua 5 trạng thái từ lúc tạo đến khi hoàn tất. Trạng thái ban đầu luôn là pending (chờ xác nhận) ngay khi đơn được tạo. Các chuyển đổi trạng thái không bị ràng buộc cứng trong code — hệ thống chấp nhận mọi giá trị hợp lệ.



Hình 11. Workflow

Workflow Tạo Đơn Hàng (Checkout)

Khi khách hàng nhấn "Đặt hàng", API Gateway thu thập thông tin từ Cart Service rồi tạo đơn mới trong Order Service. Toàn bộ quá trình diễn ra qua các bước sau:



Hình 12. Sequence Diagram - Logic tạo đơn hàng

Sau khi đơn được tạo thành công với trạng thái pending, hệ thống chuyển hướng người dùng đến trang chi tiết đơn hàng để xem lại thông tin.

2.7. Thiết kế Payment Service (Django)

2.7.1. Model

- Bảng Payment

STT	Trường dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nội dung

1	id	BigAutoField	PK, Tự tăng	Mã số định danh duy nhất cho mỗi giao dịch thanh toán.
2	order_id	IntegerField	NOT NULL	ID đơn hàng (Liên kết logic sang Order Service).
3	customer_id	IntegerField	NOT NULL	ID khách hàng (Liên kết logic sang Customer Service).
4	amount	DecimalField(12,2)	Default=0	Số tiền thực tế cần thanh toán trong giao dịch.
5	method	CharField(20)	Choices, Default='cod'	Phương thức thanh toán khách hàng lựa chọn.
6	provider	CharField(100)	Blank=True	Tên nhà cung cấp dịch vụ (VD: MoMo, VNPay, ZaloPay).
7	transaction_id	CharField(100)	Blank=True	Mã giao dịch phản hồi từ phía đối tác thanh toán.

8	status	CharField(20)	Choices, Default='initiated'	Trạng thái của quá trình giao dịch.
9	created_at	DateTimeField	auto_now_add	Thời điểm bắt đầu tạo lệnh thanh toán.
10	updated_at	DateTimeField	auto_now	Thời điểm cập nhật trạng thái giao dịch gần nhất.

Bảng 6. Bảng Payment

- Các phương thức (method)

STT	Giá trị (Key)	Nhãn hiển thị (Label)	Mô tả hình thức
1	cod	COD	Thanh toán bằng tiền mặt khi nhận hàng.
2	bank	Chuyển khoản ngân hàng	Chuyển khoản qua số tài khoản hoặc mã QR.
3	card	Thẻ tín dụng / ghi nợ	Thanh toán qua thẻ quốc tế (Visa, Mastercard) hoặc ATM.
4	wallet	Ví điện tử	Thanh toán qua các ứng dụng ví điện tử (MoMo, ShopeePay...).

Bảng 7. Các method

2.7.2. Trạng thái

Payment Service định nghĩa 5 trạng thái cho một giao dịch thanh toán, phản ánh vòng đời từ lúc khởi tạo đến khi hoàn tất hoặc hủy.

STT	Giá trị (Key)	Nhãn hiển thị	Ý nghĩa nghiệp vụ
1	initiated	Khởi tạo	Giao dịch vừa được tạo trên hệ thống, chưa bắt đầu quá trình xử lý dòng tiền.
2	paid	Đã thanh toán	Tiền đã được trừ từ tài khoản khách hàng và ghi nhận thành công vào hệ thống.
3	failed	Thất bại	Quá trình thanh toán gặp lỗi (sai thẻ, hết hạn, lỗi kết nối nhà cung cấp...).
4	refunded	Đã hoàn tiền	Giao dịch đã được thực hiện hoàn trả lại tiền cho khách hàng thành công.
5	cancelled	Đã hủy	Giao dịch bị khách hàng hoặc hệ thống chủ động hủy trước khi thực hiện thanh toán.

Bảng 8. Các trạng thái

Mọi giao dịch bắt đầu ở trạng thái initiated ngay khi được tạo. Từ đó, trạng thái có thể chuyển sang các nhánh thành công, thất bại, hoặc hủy bỏ:



Hình 13. Workflow

Trạng thái paid là trạng thái kết thúc bình thường; refunded là trạng thái đặc biệt khi có yêu cầu hoàn tiền sau khi đã thanh toán thành công.

2.7.3. API

Method	Endpoint	View Class	Chức năng
GET	/payments/	PaymentList	Lấy toàn bộ danh sách giao dịch (mới nhất trước)

Method	Endpoint	View Class	Chức năng
POST	/payments/create/	PaymentCreate	Tạo giao dịch thanh toán mới
GET	/payments/<payment_id>/	PaymentDetail	Xem chi tiết một giao dịch
PATCH	/payments/<payment_id>/status/	PaymentStatusUpdate	Cập nhật trạng thái giao dịch
GET	/payments/order/<order_id>/	OrderPayments	Lấy tất cả giao dịch theo đơn hàng
GET	/payments/customer/<customer_id>/	CustomerPayments	Lấy tất cả giao dịch theo khách hàng

Bảng 9. Bảng API

2.8. Thiết kế Shipping Service (Django)

2.8.1. Model

STT	Trường dữ liệu	Kiểu dữ liệu	Ràng buộc	Mô tả nội dung
-----	----------------	--------------	-----------	----------------

1	id	BigAutoField	PK, Tự tăng	Mã định danh duy nhất cho lô hàng vận chuyển.
2	order_id	IntegerField	NOT NULL	ID đơn hàng (Liên kết logic sang Order Service).
3	customer_id	IntegerField	NOT NULL	ID khách hàng (Liên kết logic sang Customer Service).
4	receiver_name	CharField(255)	Blank=True	Họ tên người nhận hàng thực tế.
5	phone	CharField(50)	Blank=True	Số điện thoại liên lạc của người nhận.
6	address	TextField	Blank=True	Địa chỉ chi tiết nơi giao hàng đến.
7	carrier	CharField(100)	Blank=True	Đơn vị vận chuyển (Ví dụ: GHN, GHTK, Viettel Post).
8	tracking_number	CharField(100)	Blank=True	Mã vận đơn dùng để tra cứu hành trình lô hàng.
9	status	CharField(20)	Default='pending'	Trạng thái hiện tại của quy trình giao hàng.

10	created_at	DateTimeField	auto_now_add	Thời điểm hệ thống khởi tạo lô hàng.
11	updated_at	DateTimeField	auto_now	Thời điểm cập nhật trạng thái vận chuyển gần nhất.

Bảng 10. Bảng Shipping Service

2.8.2. Trạng thái

Vòng đời của một lô hàng được phản ánh qua 06 trạng thái nghiệp vụ sau:

STT	Giá trị (Key)	Nhãn hiệu thị	Ý nghĩa nghiệp vụ
1	pending	Chờ lấy hàng	Lô hàng vừa được tạo, đang chờ shipper đến kho lấy hàng.
2	picked	Đã lấy hàng	Nhân viên vận chuyển đã nhận hàng từ kho thành công.
3	shipping	Đang giao	Hàng đang trong quá trình luân chuyển đến địa chỉ người nhận.
4	delivered	Đã giao	Khách hàng đã nhận hàng thành công và ký xác nhận.
5	failed	Giao thất bại	Giao hàng không thành công (sai địa chỉ, không liên lạc được...).

6	cancelled	Đã hủy	Lô hàng bị hủy trước khi quá trình giao nhận bắt đầu.
---	-----------	--------	---

Bảng 11. Bảng trạng thái



Hình 14. Workflow

2.8.3. API

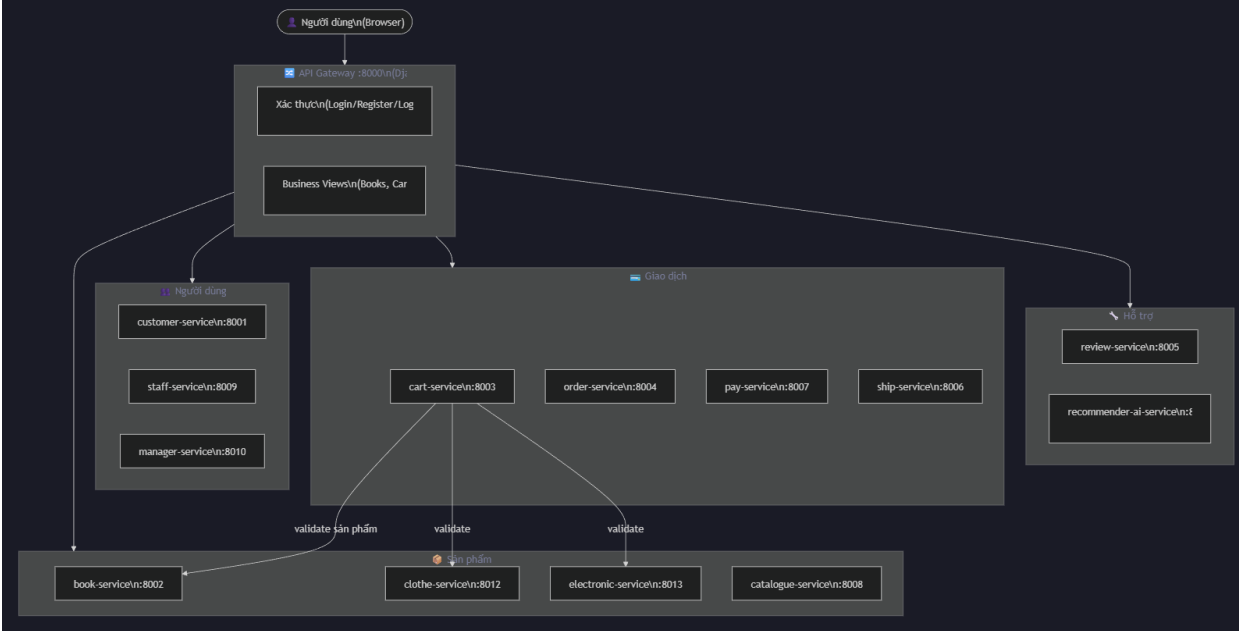
STT	Phương thức	Endpoint	View Class	Chức năng chính
1	GET	/shipments/	ShipmentList	Lấy danh sách toàn bộ lô hàng (mới nhất trước).
2	POST	/shipments/create/	ShipmentCreate	Khởi tạo một lô hàng vận chuyển mới.
3	GET	/shipments/<id>/	ShipmentDetail	Xem chi tiết thông tin và hành trình một lô hàng.
4	PATCH	/shipments/<id>/status/	ShipmentStatusUpdate	Cập nhật trạng thái mới cho lô hàng (ví dụ: shipping).
5	GET	/shipments/order/<id>/	OrderShipments	Truy xuất các lô hàng thuộc về một đơn hàng cụ thể.
6	GET	/shipments/customer/<id>/	CustomerShipments	Truy xuất toàn bộ lịch sử giao hàng của một khách

Bảng 12. Bảng API

2.9. Luồng hệ thống tổng thể

1. Kiến Trúc Tổng Quan

Hệ thống gồm **1 API Gateway** đóng vai trò cổng vào duy nhất, điều phối đến **13 microservice** backend. Mọi tương tác của người dùng đều đi qua Gateway.

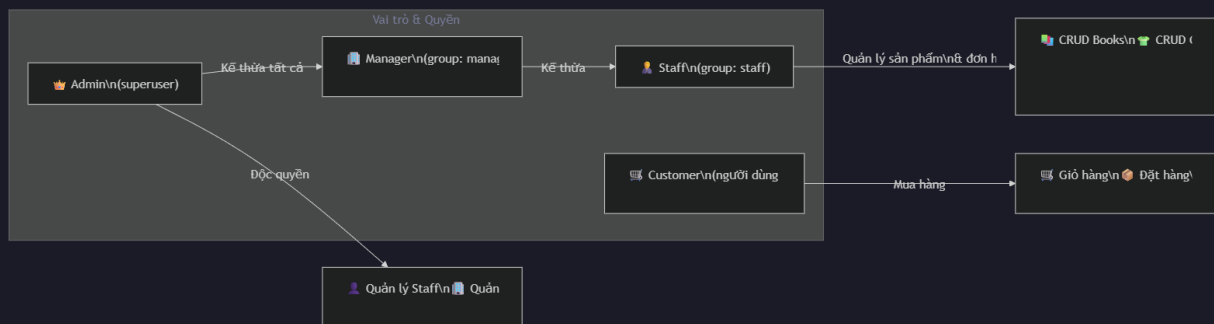


Bảng 13. Kiến trúc tổng quan

2. Phân Loại Vai Trò Người Dùng

Hệ thống có **3 vai trò** với quyền hạn khác nhau, được kiểm soát tại API Gateway qua Django Group:

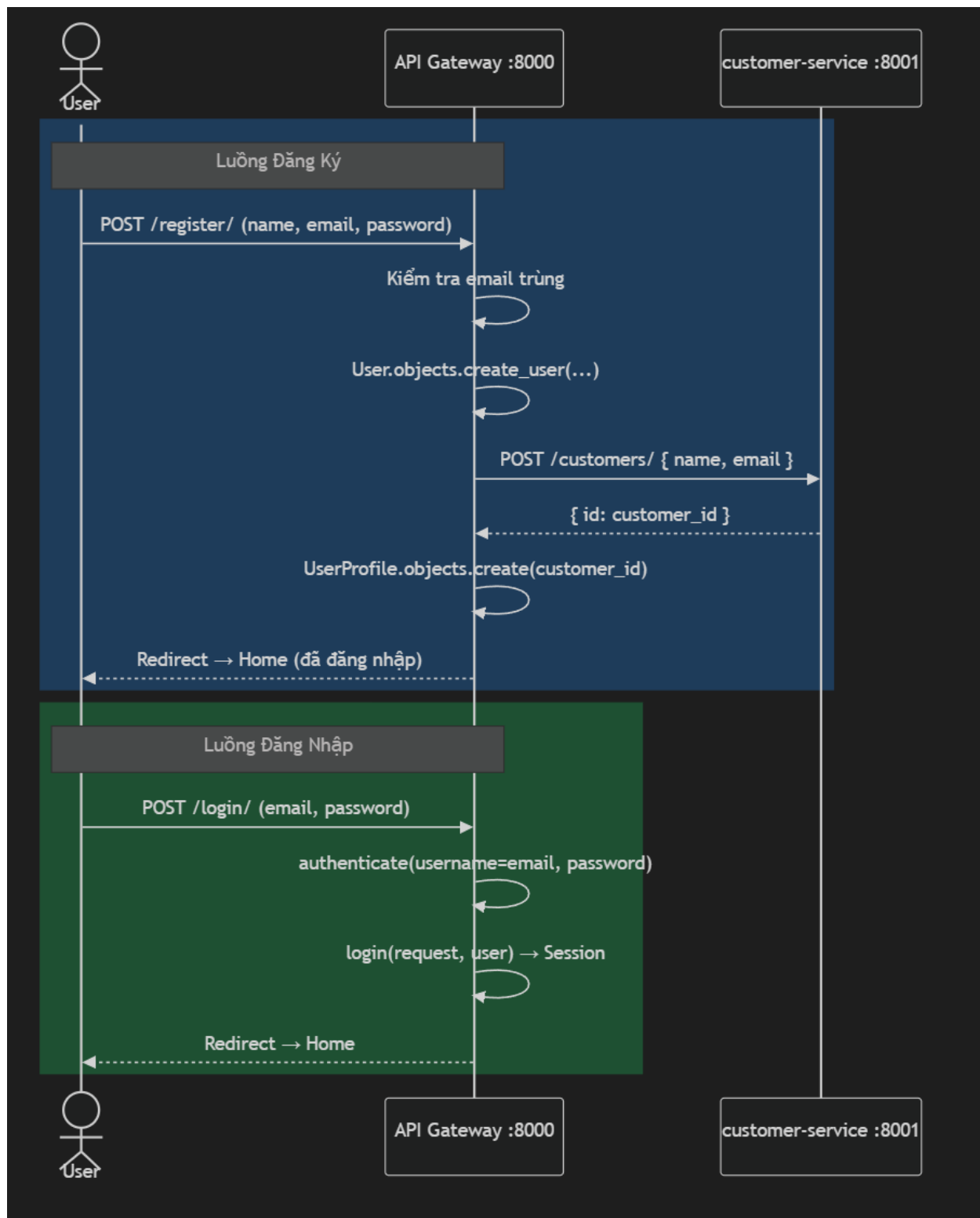
Vai trò	Nhóm Django	Quyền hạn chính
Admin	<code>superuser</code>	Toàn quyền: quản lý staff, manager, mọi dữ liệu
Manager	Group <code>manager</code>	Quản lý sách, khách hàng, xem đơn hàng
Staff	Group <code>staff</code>	Quản lý sản phẩm, xem đơn hàng
Customer	(không nhóm)	Mua hàng, xem đơn của mình



Bảng 14. Phân loại vai trò người dùng

3. Lũồng Đăng Ký & Đăng Nhập

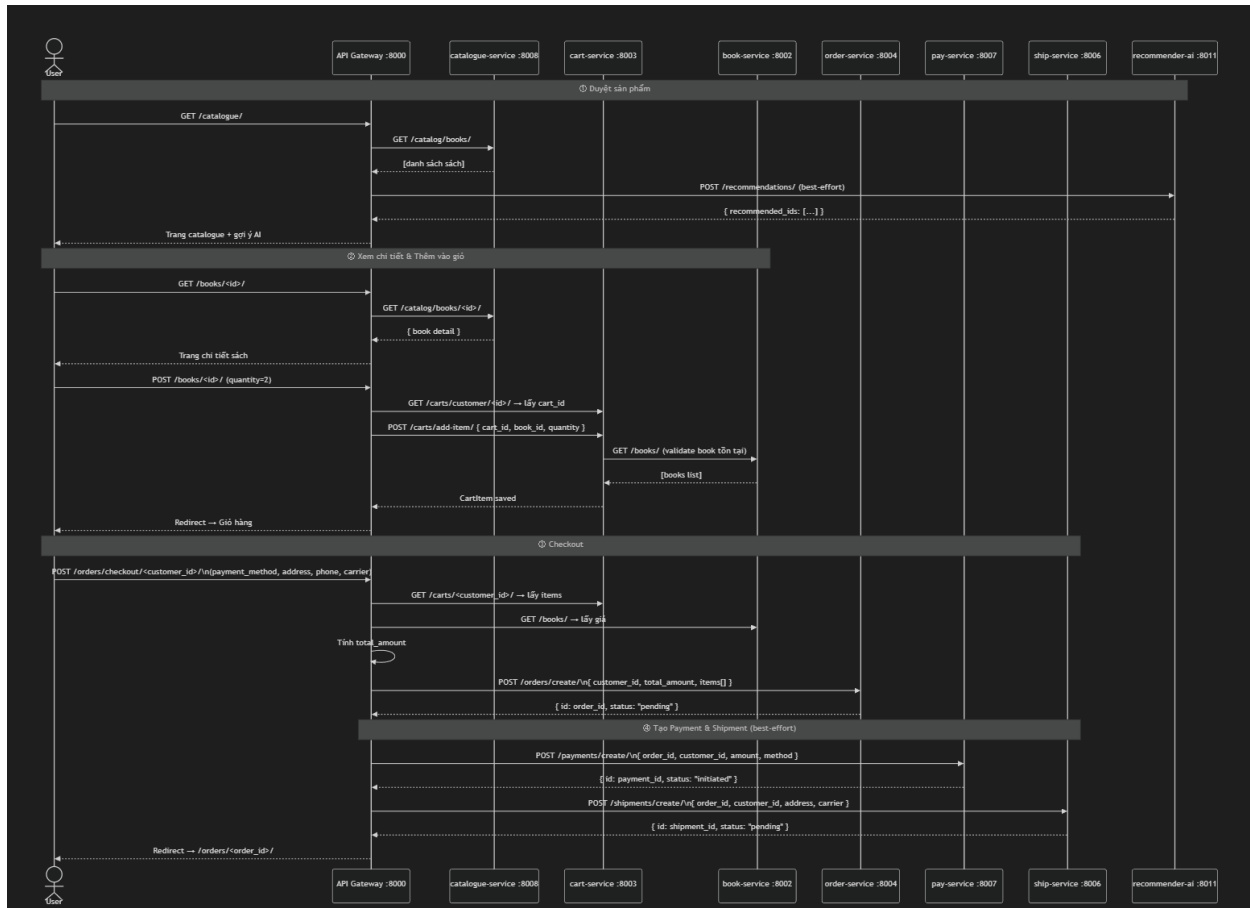
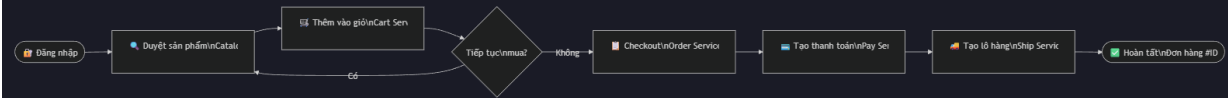
Xác thực được thực hiện hoàn toàn tại **API Gateway** (Django Session). Khi đăng ký, Gateway đồng thời tạo hồ sơ khách hàng trong `customer-service` và lưu `customer_id` vào `UserProfile` để dùng cho các luồng sau.



4. Luồng Mua Hàng Toàn Trình

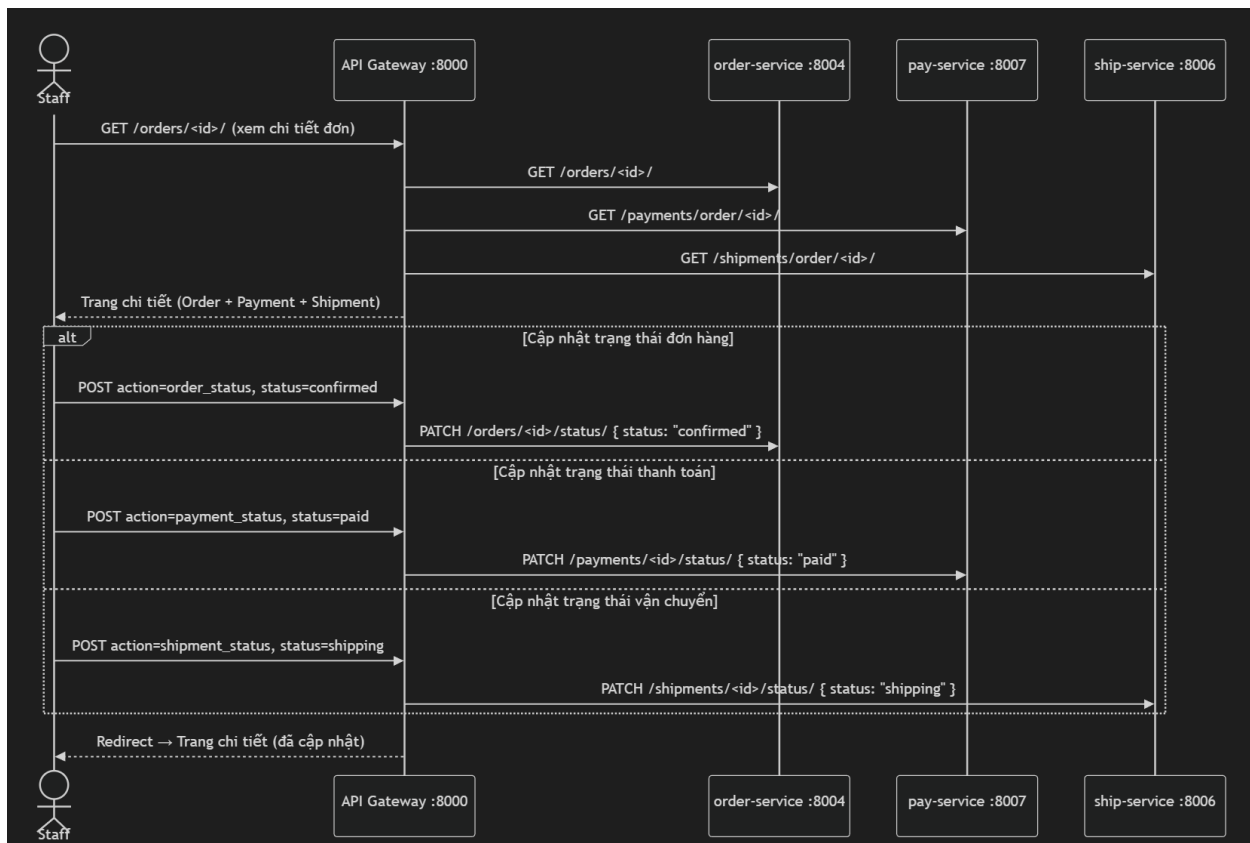
Đây là luồng nghiệp vụ cốt lõi của hệ thống, gồm 5 giai đoạn nối tiếp nhau: **Duyệt sản phẩm** → **Giỏ hàng** → **Checkout** → **Thanh toán** → **Vận chuyển**.

4.1 Sơ đồ tổng thể



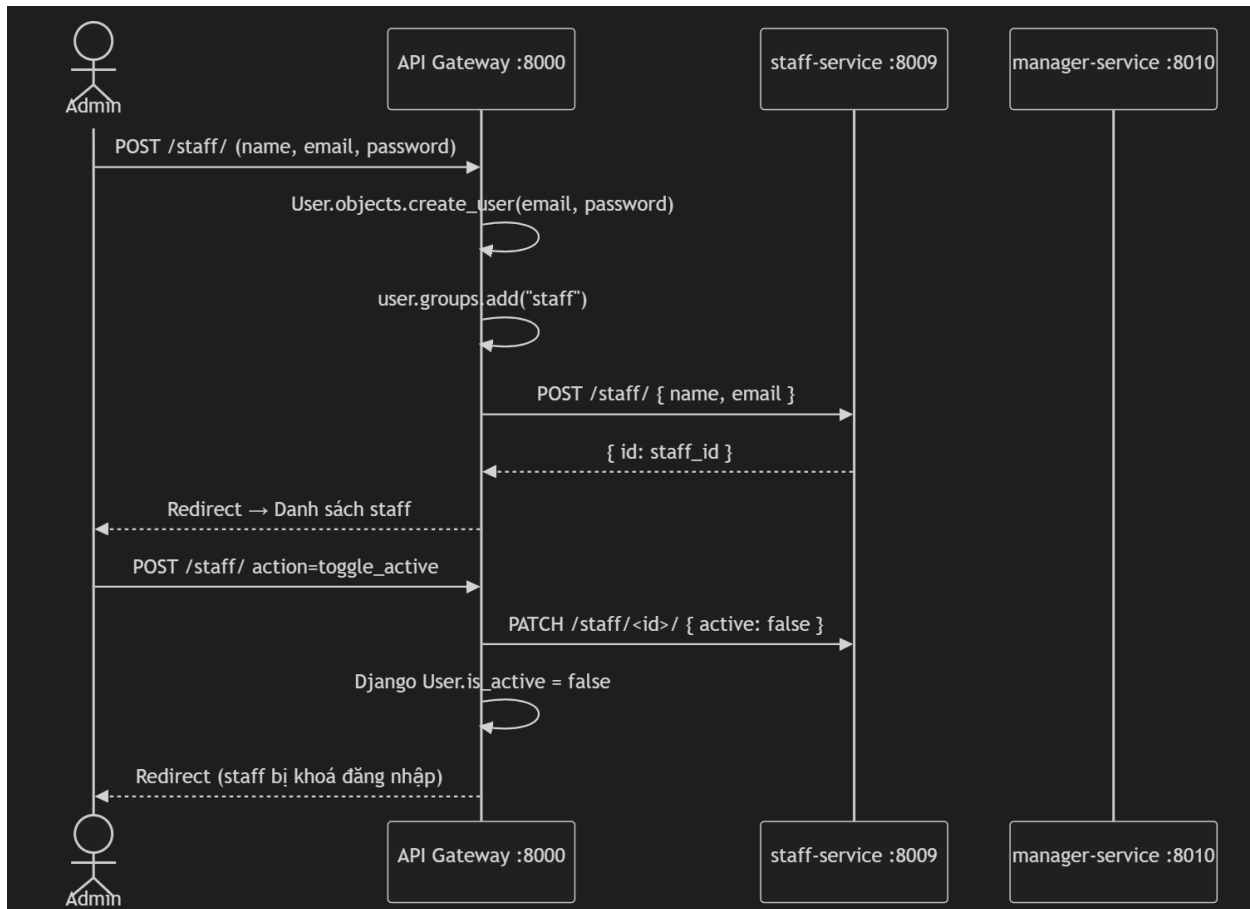
5. Luồng Cập Nhật Đơn Hàng (Staff/Manager)

Sau khi đơn được tạo, nhân viên có thể cập nhật trạng thái của **Order**, **Payment** và **Shipment** từ cùng một trang chi tiết đơn hàng.



6. Lường Quản Lý Nội Bộ (Admin)

Admin quản lý staff và manager qua API Gateway. Mỗi lần tạo nhân viên, hệ thống đồng thời tạo Django User (để đăng nhập) và record trong service tương ứng.



2.10. Hướng dẫn thực hành

2.10.1. Mục tiêu

Mục tiêu là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mục tiêu giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mục tiêu yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mục tiêu nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.10.2. Hướng dẫn vẽ Class Diagram bằng Visual Paradigm

Hướng dẫn vẽ Class Diagram bằng Visual Paradigm là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecomfinal, chủ

đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, hướng dẫn vẽ class diagram bằng visual paradigm giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, hướng dẫn vẽ class diagram bằng visual paradigm yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

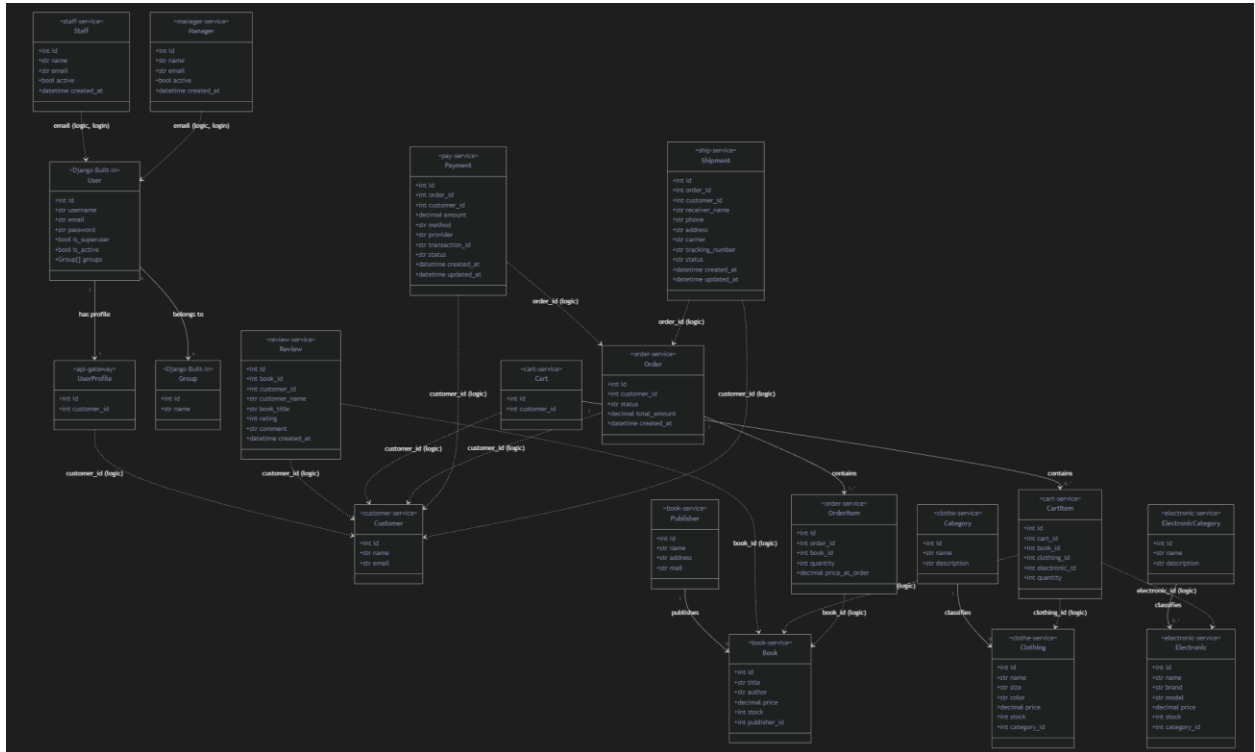
Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần hướng dẫn vẽ class diagram bằng visual paradigm nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.



Hình 15

2.10.3. Mapping Class Diagram sang Database

Mapping Class Diagram sang Database là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, mapping class diagram sang database giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, mapping class diagram sang database yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

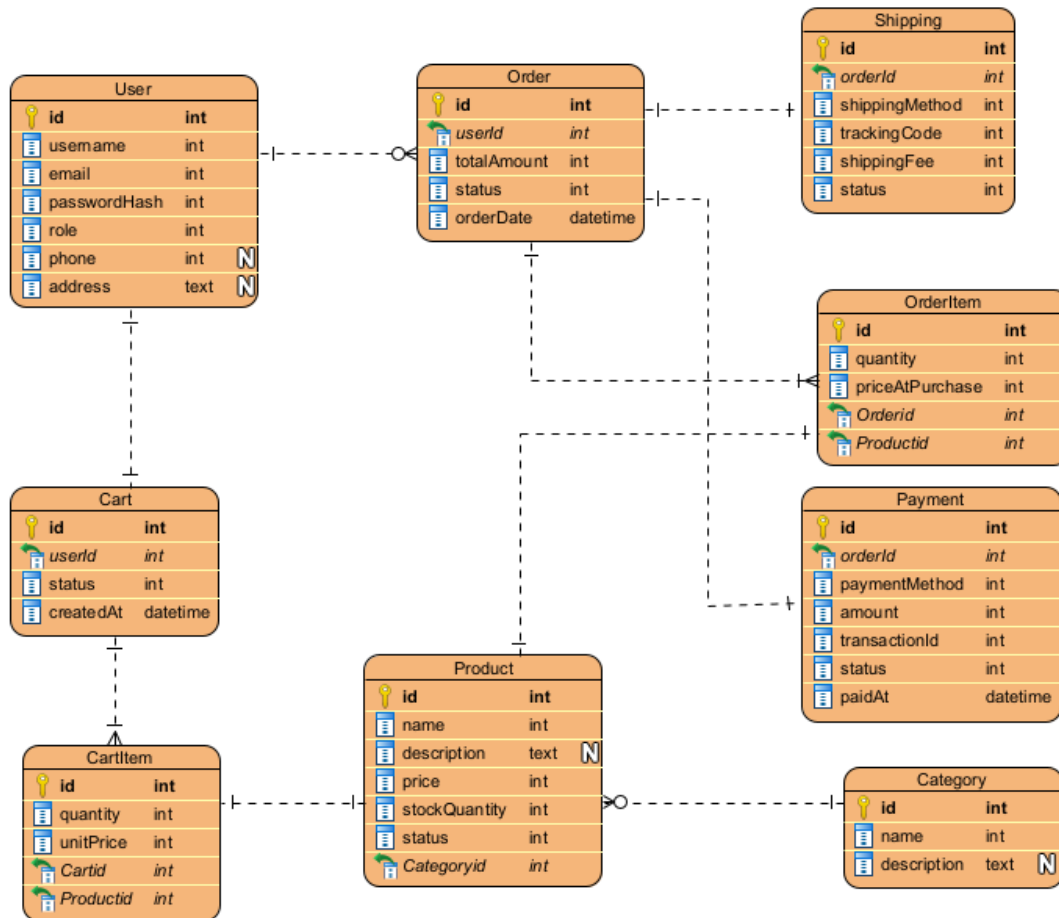
Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần mapping class diagram sang database nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.



Hình 16

2.10.4. Thiết kế Database cho từng Service

- Customer-Service

- Lưu thông tin hồ sơ khách hàng.
- CREATE TABLE app_customer (
- id INTEGER PRIMARY KEY AUTOINCREMENT,
- name VARCHAR(255) NOT NULL,
- email VARCHAR(254) NOT NULL UNIQUE
-);

- Book Service

- Lưu thông tin sách và nhà xuất bản.
- CREATE TABLE app_publisher (
- id INTEGER PRIMARY KEY AUTOINCREMENT,
- name VARCHAR(255) NOT NULL,
- address TEXT NOT NULL DEFAULT "",
- mail VARCHAR(254) NOT NULL UNIQUE
-);

```

-
- CREATE TABLE app_book (
-     id      INTEGER    PRIMARY KEY AUTOINCREMENT,
-     title   VARCHAR(255) NOT NULL,
-     author  VARCHAR(255) NOT NULL,
-     price   DECIMAL(10,2) NOT NULL,
-     stock   INTEGER     NOT NULL,
-     publisher_id INTEGER REFERENCES app_publisher(id) ON DELETE SET NULL
- );
-
- Clothes Service
-
- -- Lưu danh mục và sản phẩm quần áo.
- CREATE TABLE app_category (
-     id      INTEGER    PRIMARY KEY AUTOINCREMENT,
-     name     VARCHAR(255) NOT NULL,
-     description TEXT     NOT NULL DEFAULT ""
- );
-
- CREATE TABLE app_clothing (
-     id      INTEGER    PRIMARY KEY AUTOINCREMENT,
-     name     VARCHAR(255) NOT NULL,
-     size     VARCHAR(50) NOT NULL DEFAULT "",
-     color    VARCHAR(50) NOT NULL DEFAULT "",
-     price    DECIMAL(10,2) NOT NULL,
-     stock    INTEGER     NOT NULL,
-     category_id INTEGER REFERENCES app_category(id) ON DELETE SET NULL
- );
-
- Electronic Service
-
- -- Lưu danh mục và sản phẩm điện tử.
- CREATE TABLE app_electroniccategory (
-     id      INTEGER    PRIMARY KEY AUTOINCREMENT,
-     name     VARCHAR(255) NOT NULL,
-     description TEXT     NOT NULL DEFAULT ""
- );
-
- CREATE TABLE app_electronic (
-     id      INTEGER    PRIMARY KEY AUTOINCREMENT,
-     name     VARCHAR(255) NOT NULL,
-     brand    VARCHAR(255) NOT NULL DEFAULT "",
-     model    VARCHAR(255) NOT NULL DEFAULT "",
-     price    DECIMAL(10,2) NOT NULL,
-     stock    INTEGER     NOT NULL,
-     category_id INTEGER REFERENCES app_electroniccategory(id) ON DELETE SET NULL
- );
-
- Cart Service
-
- -- Lưu giỏ hàng theo khách hàng.
- CREATE TABLE app_cart (
-     id      INTEGER PRIMARY KEY AUTOINCREMENT,
-     customer_id INTEGER NOT NULL
- );
-

```



```

- CREATE TABLE app_cartitem (
-   id      INTEGER PRIMARY KEY AUTOINCREMENT,
-   cart_id INTEGER NOT NULL REFERENCES app_cart(id) ON DELETE CASCADE,
-   book_id  INTEGER,      -- 0 nếu không phải sách
-   clothing_id INTEGER,    -- NULL nếu không phải quần áo
-   electronic_id INTEGER,  -- NULL nếu không phải điện tử
-   quantity INTEGER NOT NULL
- );
-
- Order Service

- -- Lưu đơn hàng và chi tiết sản phẩm tại thời điểm đặt.
- CREATE TABLE app_order (
-   id      INTEGER PRIMARY KEY AUTOINCREMENT,
-   customer_id INTEGER NOT NULL,
-   status   VARCHAR(20) NOT NULL DEFAULT 'pending',
-   -- Các trạng thái: pending | confirmed | shipping | delivered | cancelled
-   total_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
-   created_at DATETIME NOT NULL
- );
-
- CREATE TABLE app_orderitem (
-   id      INTEGER PRIMARY KEY AUTOINCREMENT,
-   order_id INTEGER NOT NULL REFERENCES app_order(id) ON DELETE CASCADE,
-   book_id  INTEGER NOT NULL,
-   quantity INTEGER NOT NULL,
-   price_at_order DECIMAL(10,2) NOT NULL
- );
-
- Payment Service

- -- Lưu giao dịch thanh toán.
- CREATE TABLE app_payment (
-   id      INTEGER PRIMARY KEY AUTOINCREMENT,
-   order_id INTEGER NOT NULL,
-   customer_id INTEGER NOT NULL,
-   amount   DECIMAL(12,2) NOT NULL DEFAULT 0,
-   method    VARCHAR(20) NOT NULL DEFAULT 'cod',
-   -- Các phương thức: cod | bank | card | wallet
-   provider   VARCHAR(100) NOT NULL DEFAULT "",
-   transaction_id VARCHAR(100) NOT NULL DEFAULT "",
-   status     VARCHAR(20) NOT NULL DEFAULT 'initiated',
-   -- Các trạng thái: initiated | paid | failed | refunded | cancelled
-   created_at DATETIME NOT NULL,
-   updated_at DATETIME NOT NULL
- );
-
- Shipment Service

- -- Lưu thông tin vận chuyển lô hàng.
- CREATE TABLE app_shipment (
-   id      INTEGER PRIMARY KEY AUTOINCREMENT,
-   order_id INTEGER NOT NULL,
-   customer_id INTEGER NOT NULL,
-   receiver_name VARCHAR(255) NOT NULL DEFAULT "",
-   phone      VARCHAR(50) NOT NULL DEFAULT "",

```

```

- address TEXT NOT NULL DEFAULT "",
- carrier VARCHAR(100) NOT NULL DEFAULT "",
- tracking_number VARCHAR(100) NOT NULL DEFAULT "",
- status VARCHAR(20) NOT NULL DEFAULT 'pending',
- -- Các trạng thái: pending | picked | shipping | delivered | failed | cancelled
- created_at DATETIME NOT NULL,
- updated_at DATETIME NOT NULL
- );

```

2.10.5. So sánh MySQL vs PostgreSQL

Tiêu chí	MySQL	PostgreSQL
Phổ biến	Rất phổ biến	Rất phổ biến

JSON	Có hỗ trợ nhưng không mạnh bằng	Mạnh với JSON/JSONB
Quan hệ phức tạp	Tốt cho CRUD phổ thông	Mạnh cho truy vấn phức tạp
Use case	User/authentication	Product/order/schema phức tạp

2.10.6. Bài tập

Bài tập là nội dung cần được phân tích ở cả triển khai. Trong hệ thống ecom-final, chủ đề này với các quyết định về service boundary, database.

Khi xét theo góc nhìn nghiệp vụ, bài tập giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, bài tập yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần bài tập nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Ví dụ trong ecom-final, product-service không cần biết chi tiết thanh toán, payment-service không cần lưu mô tả sản phẩm và shipping-service không được đọc trực tiếp bảng orders. Mỗi service chỉ trao đổi qua API hoặc event đã được định nghĩa.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

2.10.7. Checklist đánh giá

Checklist đánh giá là nội dung cần được phân tích ở cả mức khái niệm và mức thiết kế triển khai. Trong hệ thống ecom-final, chủ đề này không đứng riêng lẻ mà liên kết với các quyết định về service boundary, database, API contract, bảo mật và vận hành.

Khi xét theo góc nhìn nghiệp vụ, checklist đánh giá giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, checklist đánh giá yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần checklist đánh giá nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Khi xảy ra lỗi, service phải trả response có cấu trúc rõ ràng, ví dụ code, message, trace_id và details. Trace_id giúp theo dõi request đi qua nhiều service, đặc biệt hữu ích trong debugging hệ thống phân tán.

Rủi ro phổ biến là thiết kế service theo tầng kỹ thuật thay vì theo bounded context. Cách này tạo ra các service phụ thuộc chéo và cuối cùng biến hệ thống thành distributed monolith.

2.11. Kết luận

Kết luận là nội dung cần được phân tích ở cả triển khai. Trong hệ thống ecom-final, chủ đề này với các quyết định về service boundary, database, hành.

Khi xét theo góc nhìn nghiệp vụ, kết luận giúp nhóm phát triển hiểu rõ dữ liệu nào thuộc trách nhiệm của service nào, quy tắc nào phải được giữ nhất quán và trạng thái nào cần được truyền sang service khác thông qua API hoặc sự kiện.

Khi xét theo góc nhìn kỹ thuật, kết luận yêu cầu mô tả rõ input, output, lỗi có thể xảy ra, timeout, retry, cách ghi log và cách đo lường. Đây là điểm khác biệt giữa một bản mô tả chức năng đơn giản và một tài liệu kiến trúc có thể dùng để triển khai.

Trong bài toán e-commerce, mọi thao tác đều có thể kéo theo chuỗi xử lý dài: khách hàng xem sản phẩm, thêm vào giỏ hàng, tạo đơn, thanh toán, tạo vận đơn và nhận thông báo. Vì vậy, thiết kế cần đảm bảo mỗi bước có trách nhiệm rõ ràng và không phụ thuộc trực tiếp vào database của bước khác.

Một nguyên tắc xuyên suốt là giảm coupling nhưng vẫn giữ cohesion. Service nên đủ nhỏ để triển khai độc lập, nhưng cũng phải đủ lớn để bao trọn một năng lực nghiệp vụ có ý nghĩa. Nếu tách quá nhỏ, chi phí network và vận hành sẽ vượt quá lợi ích.

Trong tài liệu thiết kế, phần kết luận nên có ví dụ cụ thể gồm endpoint, bảng dữ liệu, trạng thái, mã lỗi và tình huống kiểm thử. Các ví dụ này giúp sinh viên chứng minh rằng kiến trúc không chỉ đúng trên lý thuyết mà còn có khả năng hiện thực hóa.

Một tiêu chí đánh giá quan trọng là khả năng thay đổi trong tương lai. Khi thêm loại sản phẩm mới, phương thức thanh toán mới hoặc mô hình AI mới, thiết kế tốt sẽ cho phép mở rộng trong phạm vi service liên quan thay vì sửa toàn bộ hệ thống.

Nếu một service phụ thuộc vào dữ liệu của service khác, hệ thống nên dùng ID tham chiếu và snapshot dữ liệu cần thiết tại thời điểm giao dịch. Cách này giúp lịch sử đơn hàng không bị thay đổi khi tên sản phẩm hoặc giá hiện tại thay đổi.

Rủi ro thứ hai là thiếu quan sát hệ thống. Không có log tập trung, metric và tracing, nhóm phát triển rất khó tìm nguyên nhân khi một request checkout bị lỗi ở giữa order, payment và shipping.

3. AI Service cho tư vấn sản phẩm

3.1. Mục tiêu

Trong bối cảnh bùng nổ dữ liệu số, việc cá nhân hóa trải nghiệm khách hàng không còn là một tính năng cộng thêm mà đã trở thành yêu cầu sống còn của các hệ thống E-Commerce. AI Service được xây dựng với các mục tiêu chiến lược sau:

- Tối ưu hóa hành trình khách hàng: Giảm thời gian tìm kiếm bằng cách đưa sản phẩm phù hợp nhất lên vị trí ưu tiên.
- Tăng tỷ lệ chuyển đổi (Conversion Rate): Sử dụng các thuật toán dự báo để gợi ý các sản phẩm mà khách hàng có xác suất mua cao nhất.
- Khai phá dữ liệu ẩn: Tìm ra các mối liên hệ giữa các sản phẩm (Cross-selling) mà phương pháp thủ công không thể phát hiện.
- Tự động hóa tư vấn: Thay thế một phần đội ngũ chăm sóc khách hàng bằng Chatbot thông minh, hoạt động 24/7 với kiến thức sản phẩm sâu rộng.

3.2. Kiến trúc AI Service

AI Service được thiết kế theo mô hình Microservice độc lập, tách biệt hoàn toàn với logic nghiệp vụ của các service khác để tránh gây áp lực lên hệ thống giao dịch (Transactional System).

Cấu trúc kỹ thuật bao gồm:

- Datalake Layer: Nơi tập hợp dữ liệu thô từ các service (User, Product, Order).
- Preprocessing Pipeline: Quy trình làm sạch và chuyển đổi dữ liệu thành dạng Vector (Embedding).
- Inference Engine: Bộ máy suy diễn chứa các mô hình học máy (LSTM, GNN, RAG).
- Serving Layer: Cung cấp các Endpoint RESTful cho API Gateway gọi tới.

3.3. Thu thập dữ liệu

3.3.1. User Behavior Data

Dữ liệu này được coi là "nhiên liệu" cho AI. Hệ thống ghi lại mọi điểm chạm (Touchpoints) của người dùng:

- Implicit Feedback (Phản hồi ngầm): Lịch sử xem trang (View), thời gian dừng (Dwell time), lượt click vào sản phẩm.
- Explicit Feedback (Phản hồi tường minh): Điểm đánh giá (Rating), bình luận (Review), danh sách yêu thích (Wishlist).
- Transaction Data: Lịch sử mua hàng thực tế - đây là dữ liệu có trọng số cao nhất.

3.3.2. Ví dụ dataset

Dữ liệu thô từ các Microservices thường rời rạc và không đồng nhất, cần trải qua các bước:

- Cleaning: Loại bỏ các bản ghi trùng lặp hoặc hành vi của Bot.
- Encoding: Chuyển đổi các thuộc tính định danh (Categorical data) sang dạng số.
- Normalization: Chuẩn hóa các giá trị về cùng một khoảng $[0, 1]$ để tăng tốc độ huấn luyện.
- Sessionization: Nhóm các hành động của người dùng theo từng phiên làm việc để hiểu ngữ cảnh hiện tại.

3.4. Mô hình LSTM (Sequence Modeling)

3.4.1. Ý tưởng

LSTM (Long Short-Term Memory) là một dạng mạng nơ-ron hồi quy (RNN) có khả năng ghi nhớ thông tin trong thời gian dài. Trong E-Commerce, hành vi mua sắm là một chuỗi (Sequence). Nếu khách hàng vừa xem "Máy ảnh", sau đó xem

"Ống kính", thì sản phẩm tiếp theo họ cần nhiều khả năng là "Thẻ nhớ" hoặc "Túi đựng".

3.4.2. Model chi tiết

Mô hình sử dụng các cổng (Gates) để điều phối luồng thông tin:

- Forget Gate: Loại bỏ các thông tin cũ không còn liên quan (ví dụ: các sản phẩm khách đã xem từ 1 năm trước).
- Input Gate: Cập nhật các sản phẩm mới khách vừa quan tâm vào bộ nhớ ngắn hạn.
- Output Gate: Quyết định sản phẩm nào sẽ được gợi ý tiếp theo.

Công thức cập nhật trạng thái ô (Cell state)

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

3.5. Knowled Graph với Neo4j

3.5.1. Mô hình đồ thị

Neo4j sử dụng mô hình Property Graph, bao gồm các thành phần cơ bản sau:

- Nút (Nodes): Đại diện cho các thực thể trong hệ thống.
- Ví dụ: Khách hàng, Sản phẩm, Tác giả, Địa điểm.
- Nhãn (Labels): Được dùng để phân loại các nút. Một nút có thể có một hoặc nhiều nhãn.
- Ví dụ: Một nút có nhãn :Person hoặc :Movie.
- Mối quan hệ (Relationships): Kết nối các nút với nhau, luôn có hướng (direction) và loại (type).
- Ví dụ: (Người)-[:MUA]->(Sản phẩm), (Tác giả)-[:VIẾT]->(Sách).

- Thuộc tính (Properties): Các cặp khóa - giá trị (key-value) được lưu trữ trên cả nút và mối quan hệ để mô tả chi tiết hơn.
- Ví dụ: Nút :Person có thuộc tính name: "Nguyễn Văn A", mối quan hệ :MUA có thuộc tính ngay_mua: "2023-10-27".

Lợi ích của mô hình này:

- Sơ đồ linh hoạt (Schema-free): Dễ dàng thêm các loại thực thể hoặc mối quan hệ mới mà không cần định nghĩa lại cấu trúc bảng phức tạp.
- Hiệu suất truy vấn sâu: Khả năng duyệt qua các mối quan hệ đa tầng (multi-hop) với tốc độ không đổi ngay cả khi dữ liệu tăng trưởng lớn.

3.5.2. Ví dụ Cypher

Cypher là ngôn ngữ truy vấn mang tính khai báo của Neo4j, được thiết kế để dễ đọc và tập trung vào việc mô tả các mẫu (patterns) trong đồ thị bằng cú pháp giống như vẽ hình (ASCII art).

a. Tạo dữ liệu (Create)

Dưới đây là lệnh tạo một mẫu đơn giản gồm diễn viên và phim:

```
CREATE (keanu:Person {name: 'Keanu Reeves', born: 1964})
```

```
CREATE (matrix:Movie {title: 'The Matrix', released: 1999})
```

```
CREATE (keanu)-[:ACTED_IN {role: 'Neo'}]->(matrix)
```

b. Truy vấn dữ liệu (Match)

Tìm kiếm tất cả các phim mà diễn viên "Keanu Reeves" đã tham gia:

```
MATCH (p:Person {name: 'Keanu Reeves'})-[:ACTED_IN]->(m:Movie)
```

```
RETURN m.title, m.released
```

c. Cập nhật dữ liệu (Merge/Set)

Sử dụng MERGE để đảm bảo không tạo trùng lặp và SET để cập nhật thuộc tính:

```
MERGE (p:Person {name: 'Keanu Reeves'})
```

```
SET p.last_updated = timestamp()
```

```
RETURN p
```

3.5.3. Truy vấn gợi ý

Thế mạnh lớn nhất của Neo4j trong Knowledge Graph là khả năng thực hiện các truy vấn gợi ý dựa trên ngữ cảnh và hành vi người dùng.

Ví dụ 1: Gợi ý theo cộng tác (Collaborative Filtering)

Gợi ý những bộ phim mà bạn của người dùng thích, nhưng người dùng đó chưa xem:

```
MATCH (u:User {name: 'An'})-[:FRIEND]->(f:User)-[:LIKES]->(m:Movie)
```

```
WHERE NOT (u)-[:LIKES|WATCHED]->(m)
```

```
RETURN m.title, COUNT(*) AS recommend_score
```

```
ORDER BY recommend_score DESC
```

```
LIMIT 5
```

Ví dụ 2: Gợi ý theo đặc điểm nội dung (Content-based)

Gợi ý các phim cùng thể loại với phim mà người dùng vừa xem:

```
MATCH (u:User {name: 'An'})-[:WATCHED]->(m1:Movie)-[:IN_GENRE]->(g:Genre)<-[:IN_GENRE]-(m2:Movie)
```

```
WHERE m1 <> m2 AND NOT (u)-[:WATCHED]->(m2)
```

```
RETURN m2.title, COUNT(g) AS common_genres
```

```
ORDER BY common_genres DESC
```

```
LIMIT 5
```

3.6. RAG (Retrieval-Augmented Generation)

3.6.1. Pipeline

Một hệ thống RAG tiêu chuẩn bao gồm hai giai đoạn chính: Chuẩn bị dữ liệu (Ingestion) và Truy vấn (Inference).

Giai đoạn 1: Chuẩn bị dữ liệu

- Thu thập dữ liệu: Nạp các tài liệu thô (PDF, Word, Website, Wiki nội bộ).
- Chia nhỏ (Chunking): Cắt tài liệu thành các đoạn văn bản nhỏ (ví dụ: mỗi đoạn 500 ký tự) để đảm bảo tính tập trung và phù hợp với giới hạn ngữ cảnh của LLM.
- Nhúng (Embedding): Sử dụng các mô hình Embedding (như OpenAI text-embedding-3 hoặc HuggingFace models) để chuyển đổi các đoạn văn bản thành các vector số học (tọa độ không gian đa chiều).
- Lưu trữ: Đưa các vector này vào Vector Database.

Giai đoạn 2: Thực thi truy vấn

- Xử lý câu hỏi: Khi người dùng đặt câu hỏi, câu hỏi đó cũng được chuyển thành một vector cùng không gian với dữ liệu đã lưu.
- Truy hồi (Retrieval): Hệ thống tìm kiếm top-k (ví dụ: top 3) các đoạn văn bản có vector gần giống nhất với vector câu hỏi.

- Tăng cường (Augmentation): Kết hợp câu hỏi gốc của người dùng với các đoạn văn bản vừa tìm được để tạo thành một "Prompt" giàu ngữ cảnh.
- Tạo phản hồi (Generation): LLM đọc Prompt này và đưa ra câu trả lời dựa trên các thông tin thực tế được cung cấp.

3.6.2. Vector Database

Cơ sở dữ liệu Vector là "trái tim" của hệ thống RAG, cho phép lưu trữ và tìm kiếm dữ liệu dựa trên ý nghĩa ngữ nghĩa (semantic search) thay vì tìm kiếm từ khóa (keyword search) truyền thống.

Cơ chế hoạt động: Sử dụng các thuật toán như ANN (Approximate Nearest Neighbors) để tìm ra các điểm dữ liệu gần nhau nhất trong không gian vector. Nếu hai câu có ý nghĩa tương đương, chúng sẽ nằm gần nhau dù không dùng chung từ ngữ.

Các giải pháp phổ biến:

- Pinecone: Dịch vụ đám mây mạnh mẽ, dễ mở rộng.
- ChromaDB / FAISS: Mã nguồn mở, phù hợp cho triển khai nội bộ hoặc dự án nhỏ.
- Milvus: Được thiết kế cho dữ liệu quy mô cực lớn.
- Elasticsearch (Vector Search): Tận dụng hệ sinh thái tìm kiếm văn bản có sẵn.

3.6.3. Ví dụ

Kịch bản: Hệ thống hỗ trợ tra cứu chính sách công ty

- Thách thức: Nhân viên mới khó tìm thông tin trong hàng trăm file PDF chính sách bảo hiểm, nghỉ phép. LLM thông thường (như ChatGPT bản miễn phí) sẽ không biết chính sách riêng tư của công ty bạn.
- Giải pháp RAG:

- Dữ liệu: Toàn bộ file PDF chính sách được nhúng vào Vector DB.
- Câu hỏi: "Nếu tôi làm việc vào ngày chủ nhật, tôi được tính lương như thế nào?"
- Truy hỏi: Hệ thống tìm thấy đoạn văn trong chương III, mục 2 về "Quy định làm thêm giờ".
- Phản hồi của AI: "Dựa trên chính sách của công ty tại mục 2, chương III, làm việc vào ngày nghỉ hàng tuần (chủ nhật) sẽ được tính 200% lương cơ bản..."

Lợi ích mang lại:

- Độ chính xác cao: Trích dẫn đúng nguồn tài liệu nội bộ.
- Tiết kiệm chi phí: Không cần "Fine-tuning" lại mô hình LLM đắt đỏ mỗi khi chính sách thay đổi.
- Bảo mật: Dữ liệu có thể được lưu trữ và truy vấn trong mạng nội bộ.

3.7. Kết hợp HybridModel

Mô hình Hybrid (Mô hình lai) là sự kết hợp giữa sức mạnh của Knowledge Graph (Neo4j) và RAG (Vector Database) để khắc phục nhược điểm của từng phương pháp riêng lẻ.

- Tại sao cần kết hợp? * Vector RAG: Giỏi tìm kiếm các đoạn văn bản tương đồng nhưng đôi khi thiếu đi sự hiểu biết về cấu trúc logic và các mối quan hệ thực thể chặt chẽ (ví dụ: không biết rõ quan hệ giữa "Tác giả A" và "Công ty B").
- Knowledge Graph: Cực kỳ mạnh trong việc truy xuất dữ liệu có cấu trúc và quan hệ đa tầng, nhưng lại khó xử lý các văn bản phi cấu trúc dài.
- Cơ chế hoạt động:

- Graph-Augmented Retrieval: Khi người dùng đặt câu hỏi, hệ thống đồng thời truy vấn trên Neo4j để lấy ra các "Sự thật" (Facts) có cấu trúc và truy vấn trên Vector DB để lấy ra các "Ngữ cảnh" (Context).
- Hợp nhất dữ liệu: Kết quả từ cả hai nguồn được tổng hợp lại thành một Prompt siêu ngữ cảnh.
- Đầu ra: LLM sẽ tạo ra câu trả lời vừa chính xác về mặt dữ liệu số (từ Graph) vừa tự nhiên về mặt ngôn ngữ (từ RAG).

3.8. Hai dạng AI Service

3.8.1. Recommendation List

Đây là dạng dịch vụ AI thụ động nhưng mang tính cá nhân hóa cao, thường được hiển thị ngay trên giao diện trang chủ hoặc trang sản phẩm.

- Mục tiêu: Tăng tỷ lệ chuyển đổi (Conversion Rate) bằng cách đưa ra đúng thứ người dùng cần trước khi họ phải tìm kiếm.
- Cơ chế thực hiện:
 - Hệ thống sử dụng các thuật toán Graph Algorithms (như PageRank hoặc Similarity) trên Neo4j để xếp hạng các mục tiêu (sản phẩm/nội dung).
 - Kết quả được trả về dưới dạng một danh sách các ID đã được sắp xếp theo điểm số (Score).
- Ứng dụng: * "Sản phẩm bạn có thể thích" dựa trên lịch sử mua hàng.
 - "Những chuyên gia phù hợp" dựa trên kỹ năng và mạng lưới kết nối của người dùng.

3.8.2. Chatbot tư vấn

Đây là dạng dịch vụ AI chủ động, tương tác trực tiếp với người dùng thông qua giao diện hội thoại (Conversational UI).

- Mục tiêu: Giải đáp thắc mắc, tư vấn chuyên sâu và hướng dẫn người dùng thực hiện các tác vụ phức tạp.
- Cơ chế thực hiện:
 - Sử dụng quy trình Hybrid RAG (đã nêu ở mục 3.7) để đảm bảo câu trả lời không chỉ hay mà còn phải đúng.
 - Tích hợp bộ nhớ hội thoại (Conversation Memory) để chatbot hiểu được ngữ cảnh của các câu hỏi trước đó.
- Đặc điểm nổi bật:
 - Đa nhiệm: Vừa có thể trả lời câu hỏi chính sách, vừa có thể thực hiện hành động (ví dụ: "Hãy đặt lịch hẹn với bác sĩ X").
 - Giọng văn: Có thể tùy chỉnh để phù hợp với thương hiệu (thân thiện, chuyên nghiệp hoặc hỗ trợ kỹ thuật).

3.9. Triển khai AI Service

3.9.1. Tech stack

Để xây dựng hệ thống Hybrid RAG kết hợp Knowledge Graph, bộ công nghệ sau đây được đề xuất:

- Ngôn ngữ lập trình:
 - Python: Ngôn ngữ chủ đạo cho AI/ML nhờ hệ sinh thái thư viện phong phú.
- Framework AI & Điều phối (Orchestration):
 - LangChain / LlamaIndex: Framework giúp kết nối LLM với các nguồn dữ liệu, quản lý prompt và tạo chuỗi xử lý (chains).
- Cơ sở dữ liệu:
 - Neo4j: Lưu trữ đồ thị tri thức (Knowledge Graph) và thực hiện truy vấn quan hệ phức tạp.

- Pinecone / ChromaDB: Cơ sở dữ liệu Vector để lưu trữ và tìm kiếm ngữ nghĩa.
- Mô hình ngôn ngữ (LLMs) & Embedding:
 - OpenAI (GPT-4o, text-embedding-3): Cho hiệu suất cao nhất.
 - Hugging Face Models (Llama 3, Mistral): Lựa chọn mã nguồn mở để triển khai on-premise nhằm bảo mật dữ liệu.
- Backend & API:
 - FastAPI: Framework hiện đại, tốc độ cao để xây dựng các endpoint API phục vụ cho ứng dụng web/mobile.

3.9.2. Kiến trúc

Kiến trúc hệ thống được chia thành 3 tầng chính để đảm bảo khả năng mở rộng:

- Tầng Dữ liệu (Data Layer):
 - Dữ liệu thô được đẩy vào quy trình ETL (Extract, Transform, Load).
 - Các thực thể quan trọng được đưa vào Neo4j.
 - Nội dung chi tiết được chia nhỏ và nhúng (embedding) vào Vector Database.
- Tầng Nghiệp vụ AI (AI Logic Layer):
 - Query Parser: Phân tích câu hỏi của người dùng để quyết định xem cần lấy dữ liệu từ Graph, Vector DB hay cả hai.
 - Hybrid Retriever: Thực hiện truy vấn song song và hợp nhất kết quả.
 - Prompt Manager: Xây dựng prompt cuối cùng dựa trên dữ liệu thu thập được và gửi đến LLM.
- Tầng Giao diện (Presentation Layer):
 - Frontend (React/Next.js): Cung cấp giao diện Chatbot và các danh sách gợi ý cho người dùng cuối.

- API Gateway: Quản lý luồng yêu cầu, xác thực người dùng và giới hạn tốc độ (rate limiting).

3.10. Bài tập

Để nắm vững các khái niệm đã học, hãy thực hiện các bài tập sau:

Bài tập 1: Xây dựng mô hình đồ thị cơ bản

Sử dụng Cypher trên Neo4j Sandbox để tạo một đồ thị gồm:

- 3 Nút :Book (thuộc tính: title, author).
- 3 Nút :User (thuộc tính: name).
- Mỗi quan hệ :READ giữa User và Book với thuộc tính rating.
- Yêu cầu: Viết câu lệnh truy vấn tìm những cuốn sách mà những người bạn của "User A" đã đọc nhưng "User A" chưa đọc.

Viết một System Prompt cho chatbot đóng vai trò là "Chuyên gia tư vấn bảo hiểm".

- Yêu cầu: Prompt phải quy định rõ: Chatbot chỉ được trả lời dựa trên thông tin được cung cấp (Context), nếu không có trong dữ liệu thì phải từ chối lịch sự, không được tự ý sáng tạo thông tin.

3.11. Checklist đánh giá

Sau khi triển khai hệ thống AI Service, việc đánh giá định kỳ là bắt buộc để đảm bảo hệ thống hoạt động chính xác và an toàn. Dưới đây là bảng checklist các tiêu chí quan trọng:

- Hiệu năng kỹ thuật (Technical Performance)

[] Thời gian phản hồi (Latency): Câu trả lời từ Chatbot có được trả về trong dưới 3-5 giây không?

[] Độ chính xác của truy hồi (Retrieval Recall): Các đoạn văn bản/thực thể đề thi lấy ra có thực sự liên quan đến câu hỏi không?

[] Tính ổn định (Reliability): Hệ thống có xử lý được các truy vấn đồng thời (concurrency) mà không bị treo không?

- Chất lượng nội dung (Content Quality - RAGAS Framework)

[] Tính trung thực (Faithfulness): Câu trả lời của AI có dựa hoàn toàn vào dữ liệu cung cấp (Context) không, hay có dấu hiệu tự bịa đặt (Hallucination)?

[] Sự liên quan (Answer Relevance): Câu trả lời có đi thẳng vào vấn đề người dùng hỏi không?

[] Tính đầy đủ (Context Precision): Các thông tin từ Neo4j và Vector DB đã đủ để trả lời câu hỏi chưa?

- Dữ liệu và Kiến thức (Data & Knowledge)

[] Tính cập nhật (Data Freshness): Dữ liệu mới có được nhúng (embedding) và đồng bộ vào Graph kịp thời không?

[] Độ sạch của dữ liệu: Các nút và mối quan hệ trong Neo4j có bị trùng lặp hoặc sai lệch không?

- Trải nghiệm người dùng & Bảo mật (UX & Security)

[] Giao diện thân thiện: Người dùng có dễ dàng tương tác và hiểu các gợi ý không?

[] Bảo mật dữ liệu (Data Privacy): Thông tin nhạy cảm của người dùng có được mã hóa và bảo vệ đúng quy trình không?

[] Kiểm soát nội dung: AI có các bộ lọc để từ chối các câu hỏi độc hại hoặc không phù hợp không?

3.12. Kết luận

Chương 3 đã trình bày một cách hệ thống về việc xây dựng dịch vụ AI hiện đại thông qua sự kết hợp giữa Knowledge Graph (Neo4j) và RAG (Retrieval-Augmented Generation).

Những điểm cốt lõi đã đạt được:

- Sự kết hợp hoàn hảo: Việc sử dụng mô hình Hybrid giúp hệ thống vừa có khả năng tư duy logic theo quan hệ (từ Graph), vừa có khả năng hiểu ngữ nghĩa ngôn ngữ tự nhiên sâu sắc (từ Vector DB).
- Khắc phục nhược điểm AI: Giải pháp này giải quyết triệt để vấn đề "ảo giác" của các mô hình ngôn ngữ lớn, giúp AI trở thành một công cụ đáng tin cậy trong các lĩnh vực yêu cầu độ chính xác cao như tài chính, y tế hay pháp luật.

- Tính ứng dụng cao: Hai dạng dịch vụ là Recommendation List và Chatbot tự vấn tạo ra một hệ sinh thái hỗ trợ người dùng toàn diện, từ tìm kiếm thụ động đến tương tác chủ động.

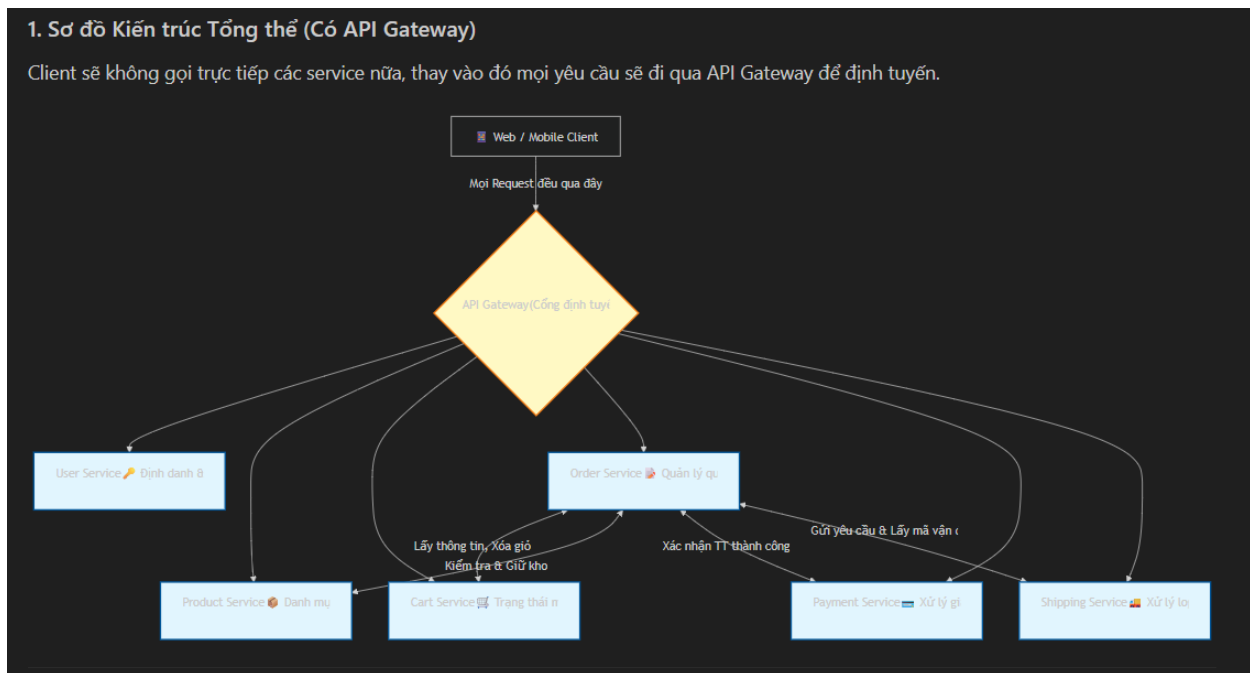
Hướng phát triển tương lai:

Trong tương lai, hệ thống có thể được nâng cấp thông qua việc tự động hóa quá trình xây dựng Graph (Auto-Graph Construction) bằng AI và tích hợp thêm các khả năng đa phương thức (Multi-modal) như xử lý hình ảnh và giọng nói để mở rộng phạm vi phục vụ khách hàng.

Tóm lại, việc làm chủ Tech-stack kết hợp giữa cơ sở dữ liệu đồ thị và kỹ thuật RAG là chìa khóa để tạo ra những dịch vụ AI thực sự thông minh, có giá trị kinh tế và khả năng thực thi cao trong thực tế doanh nghiệp.

4. Xây dựng hệ thống hoàn chỉnh

4.1. Kiến trúc tổng thể



Hình 18

4.1.1. Mô hình hệ thống

Mô hình hệ thống được thiết kế theo cấu trúc đa tầng nhằm đảm bảo sự tách biệt rõ rệt giữa các thành phần xử lý dữ liệu và giao diện người dùng. Tầng trình diễn tập trung vào việc tương tác trực tiếp với khách hàng, trong khi tầng nghiệp vụ xử lý các logic phức tạp về AI và quản lý dữ liệu. Toàn bộ hệ thống được kết nối thông qua các giao thức mạng bảo mật, tạo thành một chỉnh thể thống nhất giúp dòng thông tin luân chuyển mượt mà từ đầu vào đến đầu ra.

4.1.2. Nguyên tắc

Nguyên tắc thiết kế chủ đạo là tính module hóa và khả năng mở rộng linh hoạt. Mỗi thành phần trong hệ thống phải đảm bảo tính độc lập tối đa để khi xảy ra sự cố tại một điểm, toàn bộ hạ tầng vẫn có thể duy trì hoạt động tối thiểu. Hệ thống ưu tiên việc sử dụng các tiêu chuẩn mở và các giao diện lập trình ứng dụng tường minh,

giúp việc tích hợp thêm các công nghệ mới trong tương lai trở nên dễ dàng và ít rủi ro hơn.

4.2. System Architecture

4.2.1. Overview

Kiến trúc hệ thống cung cấp một cái nhìn toàn cảnh về cách các thành phần kỹ thuật tương tác với nhau để thực hiện các chức năng nghiệp vụ. Đây là nền tảng cốt lõi giúp đội ngũ phát triển hiểu rõ luồng đi của dữ liệu và các điểm chạm giữa các dịch vụ khác nhau. Sự phối hợp giữa phần cứng và phần mềm được tính toán kỹ lưỡng để tối ưu hóa hiệu suất xử lý, đặc biệt là đối với các tác vụ AI yêu cầu tài nguyên lớn.

4.2.2. Microservice Architecture

Hệ thống được triển khai dựa trên kiến trúc Microservice, chia nhỏ ứng dụng thành các dịch vụ độc lập có quy mô vừa phải. Mỗi dịch vụ tập trung vào một nhiệm vụ duy nhất, chẳng hạn như dịch vụ xử lý ngôn ngữ tự nhiên, dịch vụ quản lý cơ sở dữ liệu đồ thị hay dịch vụ xác thực người dùng. Cách tiếp cận này cho phép các nhóm phát triển có thể làm việc song song, đồng thời cho phép nâng cấp hoặc thay đổi công nghệ của từng phần mà không cần khởi động lại toàn bộ hệ thống.

4.2.3. API Gateway

API Gateway đóng vai trò là điểm truy cập duy nhất cho tất cả các yêu cầu từ phía khách hàng trước khi chúng được phân phối tới các dịch vụ bên trong. Thành phần này giúp đơn giản hóa việc tương tác phía client bằng cách che giấu sự phức tạp của kiến trúc microservice phía sau. Ngoài việc định tuyến, API Gateway còn chịu trách nhiệm xử lý các tác vụ xuyên suốt như ghi nhật ký, kiểm tra quyền và giới hạn lưu lượng truy cập để bảo vệ các dịch vụ nội bộ.

4.2.4. Service Communication

Việc giao tiếp giữa các dịch vụ bên trong hệ thống được thực hiện thông qua các giao thức truyền tin hiện đại như HTTP/REST cho các yêu cầu đồng bộ hoặc các hàng đợi tin nhắn cho các tác vụ bất đồng bộ. Sự lựa chọn này đảm bảo rằng các dịch vụ có thể trao đổi dữ liệu một cách tin cậy và hiệu quả. Các thông điệp được định dạng theo chuẩn JSON giúp việc giải mã và xử lý trở nên nhất quán trên nhiều ngôn ngữ lập trình khác nhau.

4.2.5. Containerization and Deployment

Toàn bộ ứng dụng được đóng gói bằng công nghệ Container, cụ thể là Docker, nhằm tạo ra môi trường vận hành biệt lập và đồng nhất. Việc này loại bỏ hoàn toàn vấn đề xung đột thư viện giữa các môi trường phát triển và thực tế. Quy trình triển khai được tự động hóa thông qua các công cụ điều phối, cho phép hệ thống tự động cập nhật phiên bản mới mà không gây gián đoạn dịch vụ, đồng thời dễ dàng khôi phục lại trạng thái cũ nếu có sai sót phát sinh.

4.2.6. System Structure

Cấu trúc hệ thống được phân định rõ ràng giữa các nhóm dịch vụ xử lý dữ liệu tĩnh và các dịch vụ tính toán động. Sự phân chia này giúp tối ưu hóa việc phân bổ tài nguyên máy chủ, nơi các dịch vụ tiêu tốn nhiều CPU cho việc suy luận AI được tách biệt khỏi các dịch vụ quản lý giao dịch thông thường. Cách tổ chức này không chỉ tăng hiệu suất mà còn giúp việc giám sát tài nguyên trở nên trực quan và chính xác hơn.

```
ecom-final/  
|-- gateway/  
|   |-- nginx.conf  
|   |-- user-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|   |-- product-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|   |-- cart-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|   |-- order-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|   |-- payment-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|   |-- shipping-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|   |-- ai-service/  
|       |-- src/  
|       |-- Dockerfile  
|       |-- requirements.txt  
|-- infrastructure/  
|   |-- docker-compose.yml  
|   |-- database/
```

Hình 19

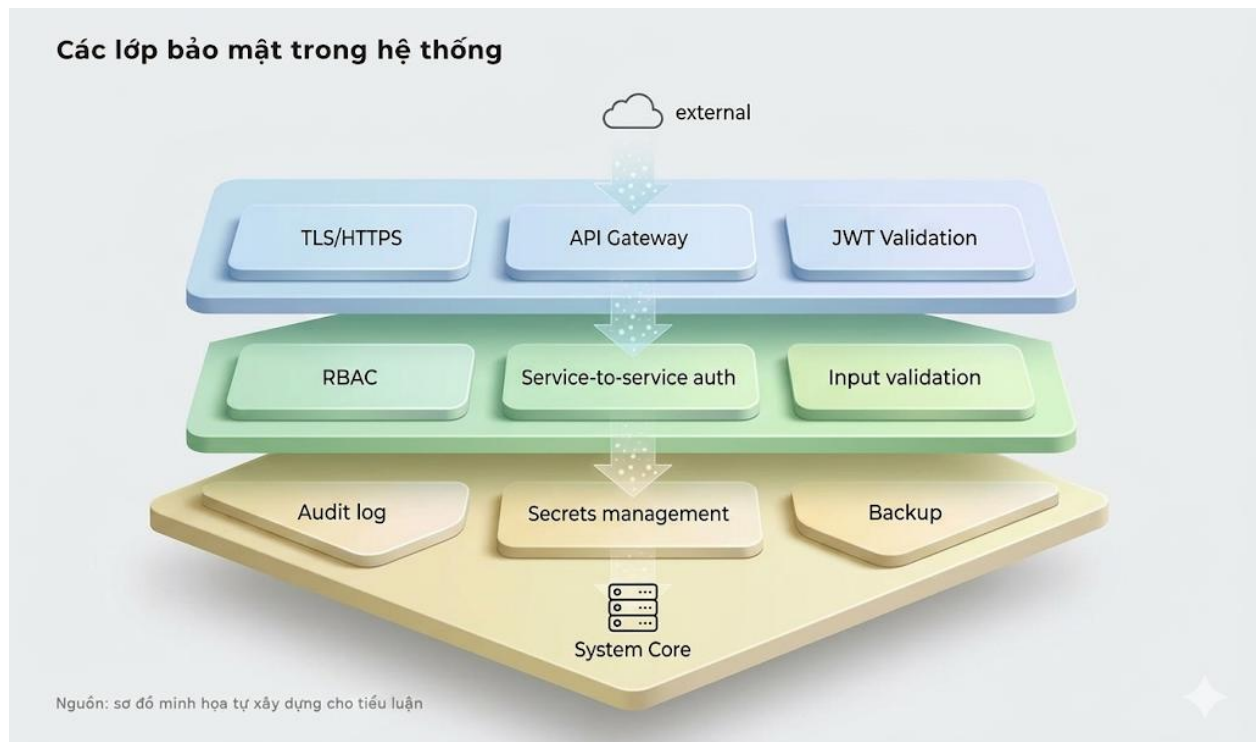
4.2.7. Design Principles

Các nguyên tắc thiết kế như tính không trạng thái (stateless) được áp dụng cho hầu hết các dịch vụ để dễ dàng mở rộng quy mô theo chiều ngang. Hệ thống cũng tuân thủ chặt chẽ nguyên tắc trách nhiệm duy nhất cho mỗi module, đảm bảo mã nguồn dễ đọc, dễ kiểm thử và bảo trì. Việc xử lý lỗi được thực hiện tập trung, giúp

hệ thống có khả năng tự phục hồi hoặc đưa ra các phản hồi lỗi rõ ràng cho người dùng cuối.

4.2.8. Security Considerations

Bảo mật là ưu tiên hàng đầu xuyên suốt kiến trúc, với việc áp dụng mã hóa dữ liệu cả khi đang lưu trữ lẫn khi đang truyền tải. Hệ thống sử dụng các cơ chế xác thực mạnh mẽ như OAuth2 kết hợp với JWT để kiểm soát truy cập ở mức độ chi tiết nhất. Các lỗ hổng bảo mật tiềm tàng được kiểm tra định kỳ thông qua các công cụ quét mã tự động, đảm bảo rằng mọi dịch vụ đều vận hành trong môi trường an toàn nhất có thể.



Hình 20

4.2.9. Discussion

Phần thảo luận tập trung vào việc phân tích những đánh đổi giữa tính phức tạp của kiến trúc microservice so với những lợi ích về khả năng mở rộng mà nó mang

lại. Mặc dù việc quản lý nhiều dịch vụ đòi hỏi hạ tầng giám sát phức tạp hơn, nhưng sự linh hoạt trong phát triển và khả năng chịu lỗi vượt trội chính là những lý do then chốt khiến mô hình này được lựa chọn cho dự án AI dài hạn.

4.3. API gateway (Nginx)

4.3.1. Vai trò

Nginx được cấu hình để thực hiện vai trò điều phối lưu lượng truy cập vô cùng quan trọng. Nó hoạt động như một Reverse Proxy, tiếp nhận các yêu cầu HTTPS từ internet và phân phối chúng đến các container dịch vụ tương ứng bên trong mạng nội bộ. Bên cạnh đó, Nginx còn đảm nhận việc cân bằng tải, giúp phân phối đều các yêu cầu đến nhiều bản sao của cùng một dịch vụ, từ đó tối ưu hóa công suất máy chủ và tránh tình trạng nghẽn cổ chai.

4.3.2. Cấu hình mẫu

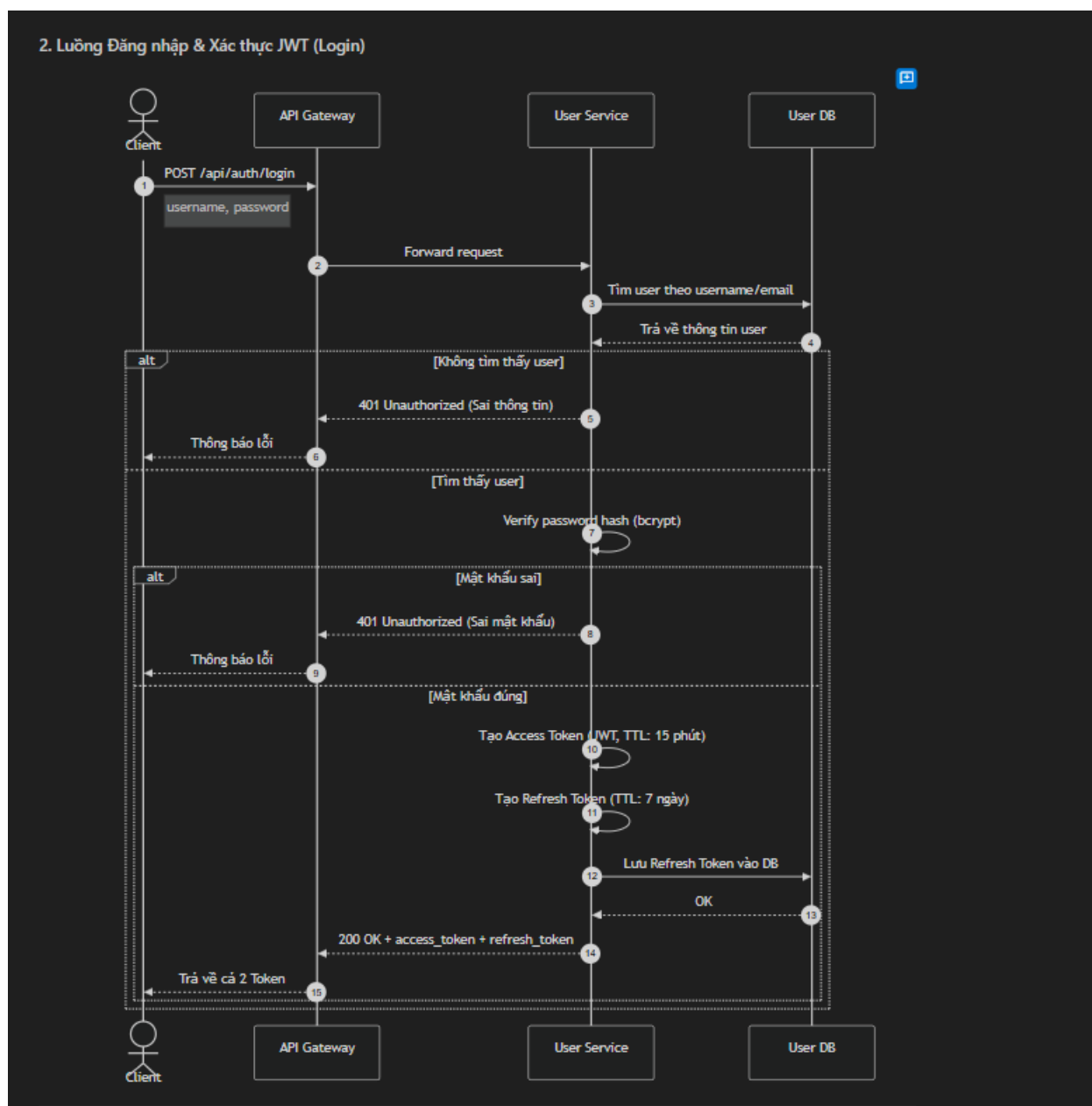
Dưới đây là một phần cấu hình mẫu của Nginx được thiết lập để điều phối yêu cầu cho hệ thống:

```
server {  
  
    listen 80;  
  
    server_name my_ai_service.com;  
  
    location /api/v1/auth {  
  
        proxy_pass http://auth_service:8080;  
  
        proxy_set_header Host $host;  
  
    }  
}
```

```
location /api/v1/ai {  
  
    proxy_pass http://ai_logic_service:5000;  
  
    proxy_set_header X-Real-IP $remote_addr;  
  
}  
  
}
```

Cấu hình này minh họa cách Nginx định tuyến các yêu cầu khác nhau đến đúng cổng của các dịch vụ nội bộ dựa trên đường dẫn API, đồng thời bảo toàn thông tin về địa chỉ IP gốc của người dùng để phục vụ cho mục đích bảo mật và phân tích.

4.4. Authentication (JWT)



Hình 21

4.4.1. Cài đặt

Cơ chế xác thực trong hệ thống được xây dựng dựa trên tiêu chuẩn JSON Web Token (JWT) để đảm bảo tính bảo mật và khả năng mở rộng. Quá trình cài đặt bắt đầu bằng việc tích hợp các thư viện chuyên dụng vào dịch vụ quản lý người dùng, cho phép hệ thống mã hóa và giải mã các token một cách an toàn. Việc cài đặt này

bao gồm cả việc thiết lập các phương thức mã hóa mạnh mẽ để bảo vệ chữ ký số của mỗi token, ngăn chặn mọi hành vi giả mạo từ bên ngoài.

4.4.2. Cấu hình

Cấu hình xác thực tập trung vào việc định nghĩa các tham số quan trọng như khóa bí mật (Secret Key), thuật toán băm và đặc biệt là thời hạn hiệu lực của token. Hệ thống được thiết lập để sử dụng cặp Access Token và Refresh Token, giúp cân bằng giữa trải nghiệm người dùng và tính bảo mật nghiêm ngặt. Ngoài ra, các cấu hình về phạm vi quyền hạn (scopes) cũng được tích hợp vào bên trong payload của token, cho phép các dịch vụ microservices khác nhau có thể kiểm tra quyền truy cập của người dùng mà không cần phải truy vấn lại cơ sở dữ liệu trung tâm.

4.4.3. Luồng

Luồng xác thực bắt đầu khi người dùng gửi thông tin đăng nhập đến hệ thống. Sau khi xác minh thành công, hệ thống sẽ cấp phát một chuỗi JWT đại diện cho phiên làm việc đó. Trong mọi yêu cầu gửi đến các dịch vụ tiếp theo, token này sẽ được đính kèm vào tiêu đề của yêu cầu. API Gateway và các dịch vụ đích sẽ thực hiện kiểm tra tính toàn vẹn và thời hạn của token trước khi cho phép thực thi các tác vụ nghiệp vụ, tạo ra một rào chắn bảo vệ xuyên suốt toàn bộ vòng đời của một yêu cầu.

4.5. Giao tiếp giữa các Service

4.5.1. REST API call

Trong kiến trúc phân tán, việc giao tiếp giữa các thành phần được thực hiện chủ yếu qua cơ chế gọi REST API đồng bộ. Mỗi dịch vụ cung cấp một tập hợp các endpoint tường minh, cho phép các dịch vụ khác truy xuất thông tin hoặc yêu cầu thực hiện hành động. Việc sử dụng giao thức HTTP giúp đơn giản hóa quá trình gỡ

lỗi và đảm bảo khả năng tương thích cao giữa các thành phần được phát triển trên các nền tảng công nghệ khác nhau.

4.5.2. Best Practice

Để đảm bảo sự ổn định khi các dịch vụ tương tác, hệ thống áp dụng các quy tắc tối ưu như thiết lập thời gian chờ (timeout) và cơ chế thử lại (retry logic) một cách thông minh. Việc xử lý lỗi được thực hiện đồng bộ trên toàn hệ thống thông qua các mã phản hồi HTTP chuẩn hóa, giúp các dịch vụ gọi có thể đưa ra quyết định xử lý phù hợp khi gặp sự cố mạng hoặc quá tải. Ngoài ra, việc giới hạn dữ liệu trả về chỉ ở những thông tin cần thiết cũng giúp tối ưu hóa băng thông truyền tải nội bộ.

4.6. Docker hóa hệ thống

4.6.1. Dockerfile (Django)

Dịch vụ chính dựa trên framework Django được đóng gói thông qua một Dockerfile tối ưu, bắt đầu từ một hình ảnh nền Python gọn nhẹ. Quy trình đóng gói bao gồm việc cài đặt các phụ thuộc cần thiết, thiết lập biến môi trường và sao chép mã nguồn vào bên trong container. Dockerfile được thiết kế theo cấu trúc nhiều tầng (multi-stage build) để giảm thiểu kích thước hình ảnh cuối cùng, đồng thời tách biệt rõ ràng giữa giai đoạn xây dựng và giai đoạn vận hành để nâng cao tính bảo mật.

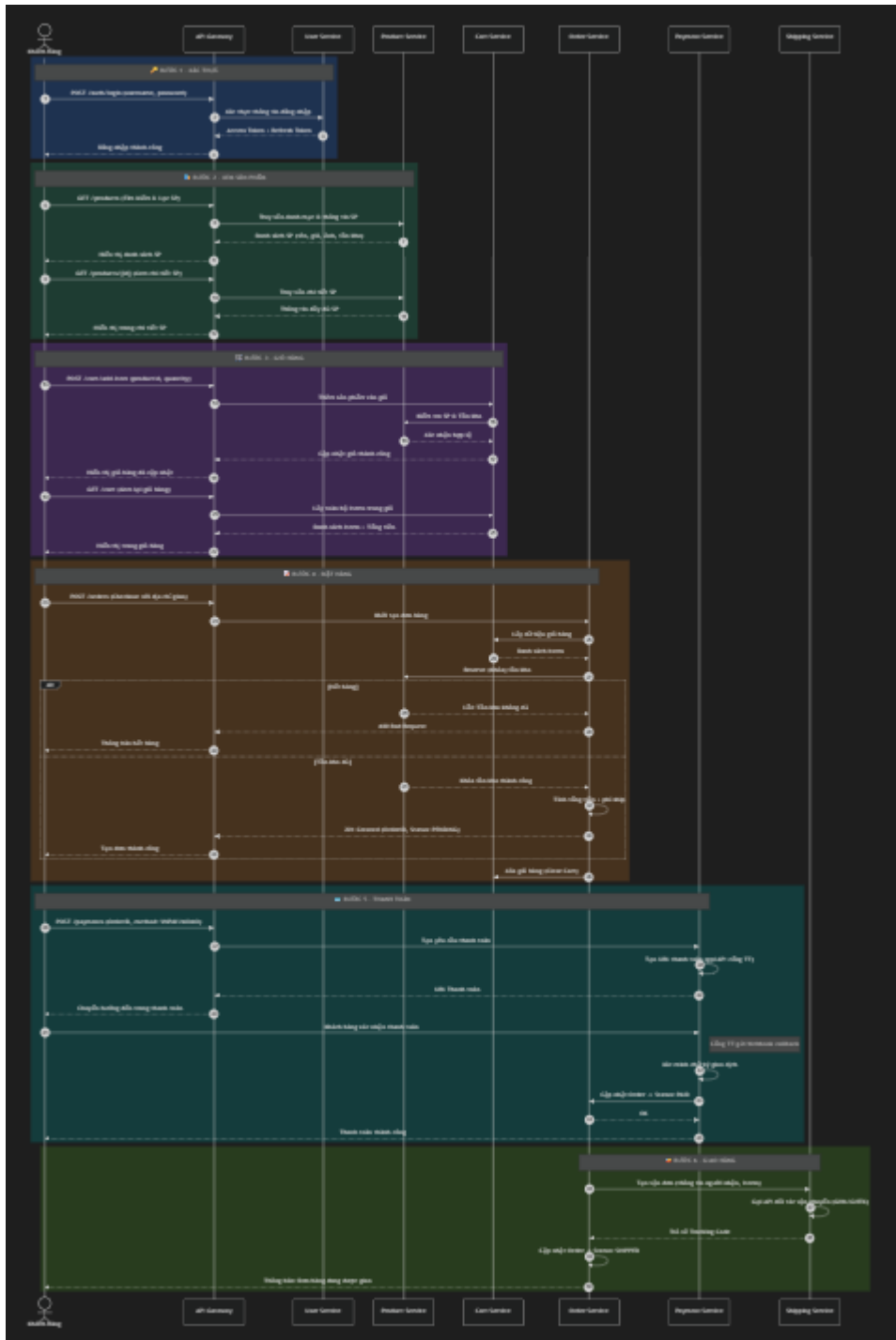
4.6.2. Docker-compose.yml

Việc điều phối toàn bộ các thành phần như Nginx, dịch vụ Django, và các cơ sở dữ liệu được thực hiện thông qua tệp cấu hình docker-compose. Tệp này định nghĩa các liên kết mạng nội bộ, cho phép các dịch vụ nhận biết và giao tiếp với nhau bằng tên định danh thay vì địa chỉ IP thay đổi. Cấu hình này cũng quản lý việc gắn các ổ đĩa dữ liệu (volumes) để đảm bảo tính bền vững của dữ liệu và thiết lập các chính sách tự động khởi động lại, giúp hệ thống luôn ở trạng thái sẵn sàng cao nhất.

4.7. Luồng hệ thống (End-to-End)

4.7.1. Use case: Mua hàng

Luồng vận hành thực tế được minh họa qua kịch bản người dùng thực hiện giao dịch mua hàng trên hệ thống. Bắt đầu từ việc chọn sản phẩm, yêu cầu được gửi qua API Gateway để xác thực người dùng, sau đó thông tin được chuyển đến dịch vụ giỏ hàng và dịch vụ kho để kiểm tra tình trạng hàng hóa. Sau khi thanh toán được xác nhận thông qua tích hợp bên thứ ba, dịch vụ đơn hàng sẽ cập nhật trạng thái và kích hoạt quy trình giao hàng. Toàn bộ quá trình diễn ra một cách liên mạch, đi qua nhiều dịch vụ nhỏ nhưng vẫn đảm bảo tính nhất quán của dữ liệu.



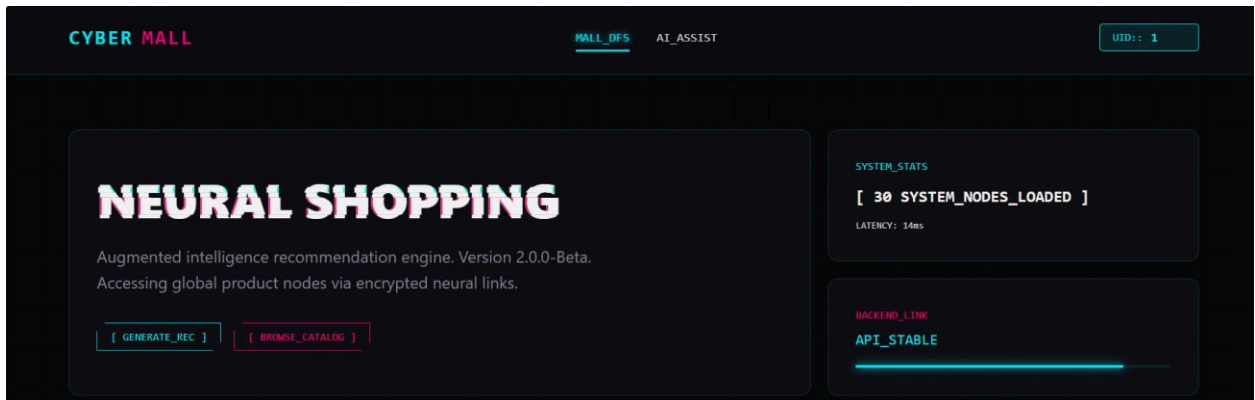
Hình 22

4.7.2. Sequence logic

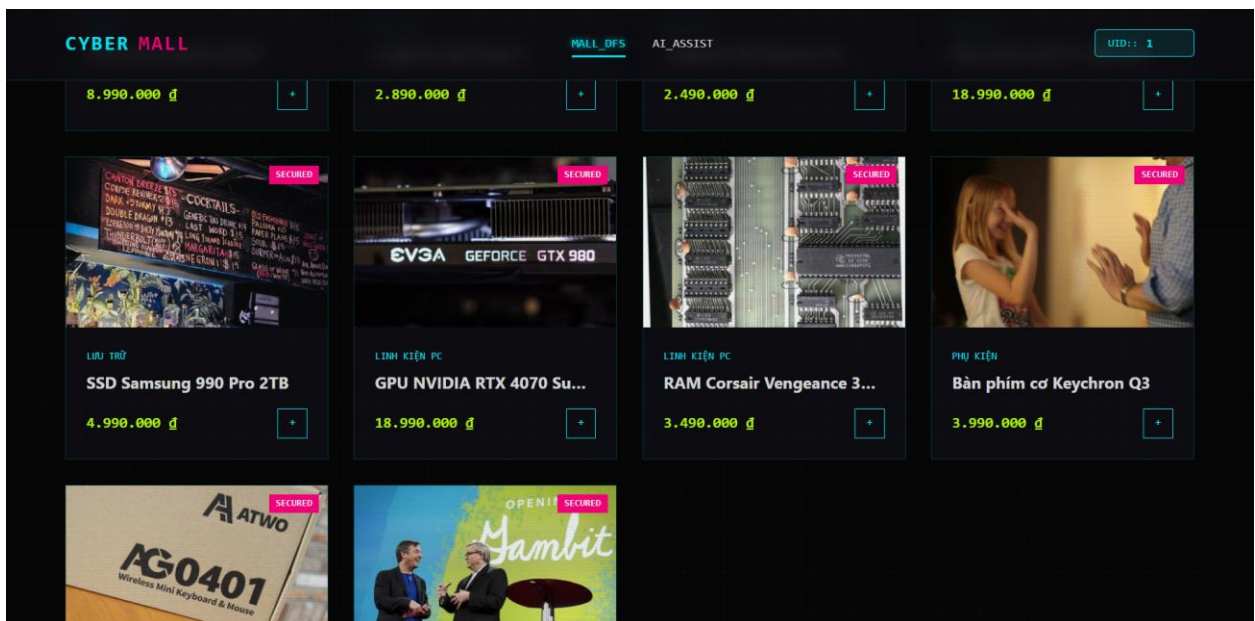
Logic tuần tự của hệ thống mô tả cách thức các thông điệp được chuyển tiếp qua lại giữa các thành phần theo thời gian thực. Mỗi bước trong quy trình đều có

sự phản hồi xác nhận để đảm bảo không có bước nào bị bỏ sót hoặc xử lý sai lệch. Sự phối hợp nhịp nhàng giữa API Gateway điều phối, dịch vụ nghiệp vụ xử lý và cơ sở dữ liệu lưu trữ tạo nên một chu trình khép kín, giúp hệ thống có thể xử lý các yêu cầu phức tạp của người dùng một cách chính xác và hiệu quả.

4.7.3. Giao diện



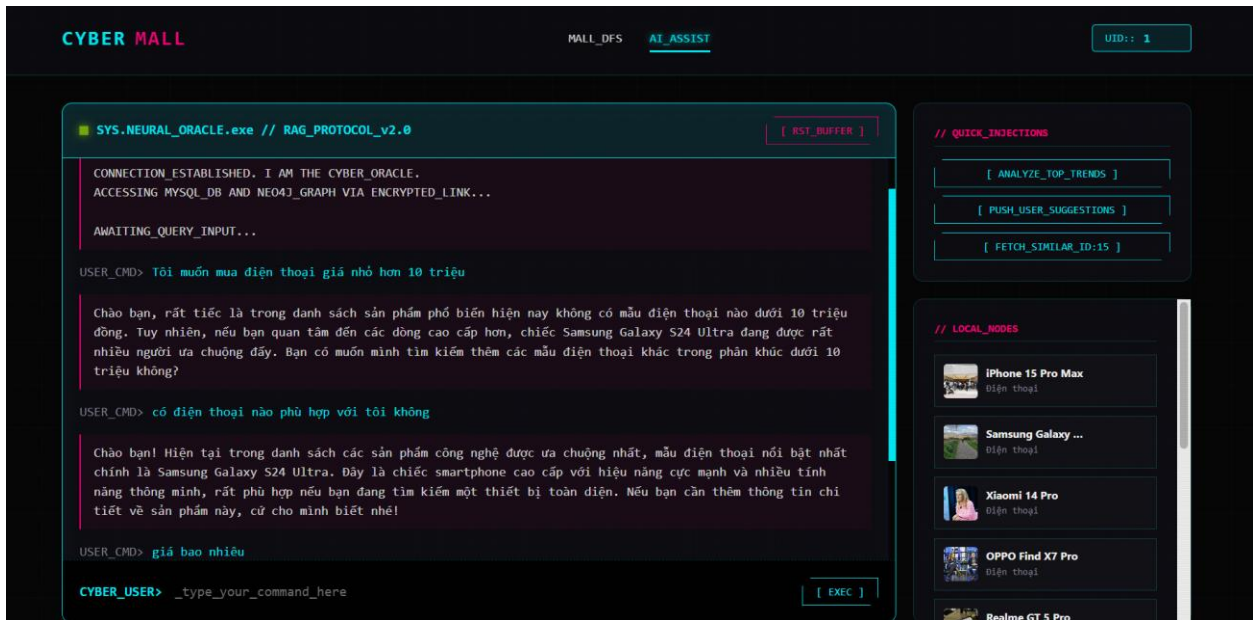
Hình 23. GD1



Hình 24. GD2



Hình 25. GD3



Hình 26. GD4

4.8. Triển khai Kubernetes (Optional)

4.8.1. Deployment

Đối với các hệ thống yêu cầu quy mô lớn và tính sẵn sàng cao, việc chuyển đổi từ Docker Compose sang Kubernetes là một bước đi tất yếu. Trong cấu hình Deployment, hệ thống định nghĩa trạng thái mong muốn cho các nhóm container (Pods), bao gồm số lượng bản sao cần chạy và chiến lược cập nhật phiên bản. Cơ chế này cho phép Kubernetes tự động phát hiện và thay thế các container bị lỗi, đảm bảo dịch vụ luôn hoạt động ổn định mà không cần can thiệp thủ công. Việc quản lý tài nguyên như CPU và bộ nhớ cũng được thiết lập chi tiết trong phần này để tối ưu hóa hiệu suất trên toàn cụm máy chủ.

4.8.2. Service

Thành phần Service trong Kubernetes đóng vai trò là một lớp trừu tượng giúp định danh và kết nối các Pods với nhau hoặc với thế giới bên ngoài. Thay vì truy cập trực tiếp qua địa chỉ IP của từng Pod vốn thường xuyên thay đổi, các thành phần

khác sẽ tương tác thông qua một tên dịch vụ duy nhất và ổn định. Service cũng tích hợp sẵn khả năng cân bằng tải nội bộ, giúp phân phối các yêu cầu đến các Pods đang hoạt động một cách hiệu quả, tạo ra một hạ tầng mạng linh hoạt và tin cậy cho kiến trúc Microservices.

4.9. Logging và Monitoring

Việc duy trì tính ổn định của một hệ thống phân tán đòi hỏi một cơ chế giám sát và ghi nhật ký toàn diện. Hệ thống thực hiện việc thu thập nhật ký hoạt động từ tất cả các dịch vụ thành viên về một kho lưu trữ tập trung, cho phép đội ngũ vận hành có thể truy vấn và phân tích nguyên nhân gốc rễ của các sự cố một cách nhanh chóng. Song song với đó, các chỉ số về hiệu suất như thời gian phản hồi, tỉ lệ lỗi và mức độ tiêu thụ tài nguyên được theo dõi theo thời gian thực thông qua các bảng điều khiển trực quan. Hệ thống cảnh báo tự động cũng được thiết lập để thông báo ngay lập tức khi có bất kỳ chỉ số nào vượt ngưỡng an toàn, giúp ngăn ngừa rủi ro trước khi chúng ảnh hưởng đến người dùng cuối.

4.10. Đánh giá hệ thống

4.10.1. Hiệu năng

Đánh giá về hiệu năng cho thấy hệ thống có khả năng xử lý các truy vấn AI phức tạp với độ trễ ở mức chấp nhận được đối với người dùng phổ thông. Việc áp dụng cơ chế bộ đệm và tối ưu hóa các câu lệnh truy vấn cơ sở dữ liệu đồ thị đã giúp giảm thiểu đáng kể thời gian phản hồi. Tuy nhiên, hiệu năng thực tế vẫn phụ thuộc nhiều vào cấu hình phần cứng của máy chủ chạy các mô hình ngôn ngữ lớn, đòi hỏi sự cân bằng giữa chi phí tài nguyên và tốc độ xử lý.

4.10.2. Khả năng mở rộng

Kiến trúc Microservices kết hợp với công nghệ Containerization mang lại khả năng mở rộng vượt trội cho hệ thống. Khi lưu lượng truy cập tăng cao, việc tăng

cường số lượng bản sao của các dịch vụ quan trọng được thực hiện một cách nhanh chóng mà không làm gián đoạn các phần khác. Cấu trúc này cũng cho phép mở rộng quy mô theo cả chiều dọc bằng cách nâng cấp tài nguyên máy chủ hoặc theo chiều ngang bằng cách thêm mới các nút vào cụm dịch vụ một cách linh hoạt.

4.10.3. Ưu điểm

Ưu điểm lớn nhất của hệ thống nằm ở tính module hóa cao, giúp quá trình phát triển và bảo trì trở nên đơn giản và ít rủi ro. Việc tách biệt các dịch vụ cho phép áp dụng các công nghệ tối ưu nhất cho từng tác vụ chuyên biệt, đồng thời tăng cường khả năng chịu lỗi của toàn bộ hạ tầng. Sự tích hợp chặt chẽ giữa AI và cơ sở dữ liệu đồ thị cũng tạo ra một lợi thế cạnh tranh về độ chính xác và tính thông minh của các phản hồi so với các hệ thống truyền thống.

4.10.4. Nhược điểm

Mặc dù có nhiều ưu điểm, hệ thống vẫn đối mặt với sự phức tạp trong việc quản lý và điều phối một số lượng lớn các dịch vụ nhỏ. Chi phí hạ tầng và yêu cầu về băng thông mạng nội bộ cũng cao hơn so với mô hình nguyên khối. Ngoài ra, việc duy trì tính nhất quán dữ liệu giữa các dịch vụ phân tán vẫn là một thách thức kỹ thuật đòi hỏi các chiến lược xử lý đồng bộ hóa và quản lý giao dịch phức tạp hơn.

4.11. Bài tập thực hành

Nhằm củng cố kiến thức về triển khai hệ thống, sinh viên cần thực hiện bài tập xây dựng một Dockerfile cho dịch vụ AI xử lý ảnh và tích hợp nó vào tệp docker-compose hiện có. Bài tập yêu cầu thiết lập các biến môi trường phù hợp, cấu hình ánh xạ cổng và kiểm tra khả năng giao tiếp giữa dịch vụ mới này với API Gateway. Ngoài ra, việc viết một đoạn mã cấu hình Nginx để định tuyến yêu cầu cho dịch vụ mới cũng là một phần quan trọng để hiểu rõ luồng đi của dữ liệu trong kiến trúc thực tế.

4.12. Checklist đánh giá

Bảng checklist đánh giá cuối cùng tập trung vào các tiêu chí hoàn thiện của kiến trúc bao gồm việc kiểm tra tính toàn vẹn của tệp cấu hình docker-compose và độ an toàn của các khóa bí mật trong JWT. Các dịch vụ phải đảm bảo khởi động đúng thứ tự và có khả năng tự phục hồi khi bị tắt đột ngột. Cuối cùng, việc kiểm tra khả năng truy cập API từ bên ngoài thông qua Nginx với các tiêu đề bảo mật đúng quy chuẩn là bước chốt chặn để xác nhận hệ thống đã sẵn sàng cho giai đoạn vận hành thực tế.

BÁO CÁO ĐÁNH GIÁ TÀI LIỆU TIỂU LUẬN

Chủ đề: Kiến trúc Microservices và Hệ thống E-Commerce tích hợp AI Service

I. Mở đầu

Tài liệu tiểu luận được xây dựng với định hướng nghiên cứu và thiết kế hệ thống E-Commerce theo kiến trúc Microservices, đồng thời tích hợp thêm các thành phần AI Service nhằm hỗ trợ tư vấn sản phẩm, chatbot và Recommendation System. Đây là một chủ đề mang tính thực tiễn cao, phù hợp với xu hướng phát triển phần mềm hiện đại khi các doanh nghiệp lớn đang dần chuyển đổi từ kiến trúc Monolithic sang Microservices nhằm nâng cao khả năng mở rộng, triển khai và bảo trì hệ thống.

Bài tiểu luận không chỉ dừng lại ở mức mô tả lý thuyết mà còn thể hiện được tư duy phân tích hệ thống, phân rã nghiệp vụ, thiết kế service và xây dựng mô hình dữ liệu tương đối đầy đủ. Nội dung tài liệu bao gồm nhiều thành phần quan trọng như:

- So sánh Monolithic và Microservices
- Thiết kế hệ thống E-Commerce
- Phân rã service theo bounded context
- Thiết kế database cho từng service
- API và trạng thái nghiệp vụ
- AI Service với LSTM, Knowledge Graph và RAG
- Triển khai hệ thống bằng Django và các công nghệ container hóa

Nhìn chung, tài liệu cho thấy có sự đầu tư đáng kể về mặt nghiên cứu kiến trúc phần mềm hiện đại, đồng thời tiếp cận đúng các xu hướng đang được sử dụng trong thực tế.

II. ĐÁNH GIÁ CHƯƠNG 1 – TỔNG QUAN KIẾN TRÚC MICROSERVICES

1. So sánh Monolithic và Microservices

Một trong những điểm tích cực của tài liệu là đã trình bày được các đặc trưng cơ bản giữa hai mô hình kiến trúc Monolithic và Microservices. Em đã phân tích tương đối rõ:

- Mức độ coupling
- Khả năng triển khai
- Tính mở rộng
- Quản lý cơ sở dữ liệu
- Fault isolation
- Độ phức tạp hệ thống

Các nội dung này giúp người đọc có cái nhìn tổng quan về lý do tại sao Microservices ngày càng được áp dụng phổ biến trong các hệ thống quy mô lớn.

Tài liệu cũng thể hiện được tư duy đúng khi nhấn mạnh rằng Microservices không đơn giản là “chia nhỏ hệ thống”, mà cần phải đảm bảo:

- Tính độc lập của service
- Database per service
- Bounded context
- Loose coupling
- Khả năng triển khai riêng biệt

Đặc biệt, em có đề cập đến nguy cơ “Distributed Monolith”, đây là một khái niệm tương đối quan trọng trong thiết kế Microservices hiện đại. Việc đưa được nội dung này vào tài liệu cho thấy em có tìm hiểu sâu hơn so với mức kiến thức cơ bản.

Tuy nhiên, phần này vẫn còn một số hạn chế:

- Chưa có nhiều hình minh họa trực quan
- Chưa mô tả cụ thể kiến trúc deployment
- Chưa phân tích sâu các khó khăn của distributed system như network latency hoặc eventual consistency

Dù vậy, xét ở phạm vi tiểu luận học phần, phần trình bày hiện tại vẫn đạt yêu cầu tương đối tốt.

2. Phân tích tư duy kiến trúc

Tài liệu thể hiện tương đối rõ tư duy thiết kế hệ thống theo domain và bounded context. Đây là một điểm mạnh đáng ghi nhận bởi nhiều bài tiểu luận thường chỉ tập trung vào code mà thiếu đi phần tư duy kiến trúc.

Các nguyên tắc được áp dụng gồm:

- Mỗi service có trách nhiệm riêng
- Không truy cập trực tiếp database của service khác
- Giao tiếp thông qua API hoặc event
- Giảm coupling nhưng vẫn đảm bảo cohesion

Đây đều là những nguyên tắc quan trọng trong kiến trúc Microservices thực tế.

Ngoài ra, tài liệu cũng có đề cập:

- Retry
- Timeout
- Logging
- Tracing
- Snapshot dữ liệu

Điều này cho thấy em không chỉ tiếp cận hệ thống ở góc nhìn lập trình mà còn chú ý tới khía cạnh vận hành và quan sát hệ thống.

3. Ví dụ Healthcare Microservices

Việc đưa thêm ví dụ Healthcare giúp tài liệu có tính mở rộng và không bị giới hạn trong phạm vi E-Commerce. Em đã trình bày được:

- Cách phân rã nghiệp vụ
- Event flow
- Quan hệ giữa các service

Nội dung này góp phần chứng minh rằng em hiểu được cách áp dụng Microservices vào nhiều bài toán khác nhau.

Tuy nhiên, phần Healthcare vẫn thiên về mô tả khái niệm và chưa có nhiều minh họa bằng code hoặc UML diagram.

III. ĐÁNH GIÁ CHƯƠNG 2 – THIẾT KẾ HỆ THỐNG E-COMMERCE

1. Phân tích yêu cầu hệ thống

Em đã xác định được khá đầy đủ các thành phần chính của một hệ thống E-Commerce:

- Khách hàng
- Sản phẩm
- Giỏ hàng
- Đơn hàng
- Thanh toán
- Vận chuyển

Đồng thời tài liệu cũng phân biệt được:

- Functional Requirements
- Non-functional Requirements

Đây là bước cần thiết trước khi tiến hành thiết kế hệ thống.

Ngoài ra, các trạng thái nghiệp vụ như:

- Payment status
- Shipment status
- Order status

được mô tả tương đối rõ ràng, giúp hệ thống có logic xử lý nhất quán hơn.

2. Phân rã Microservices

Một điểm mạnh đáng ghi nhận là em đã phân rã hệ thống thành nhiều service tương đối hợp lý:

- Customer Service
- Product Service
- Cart Service

- Order Service
- Payment Service
- Shipment Service

Cách phân chia này phù hợp với mô hình bounded context và phản ánh đúng nghiệp vụ thực tế của hệ thống thương mại điện tử.

Đặc biệt:

- Payment Service không phụ thuộc trực tiếp vào Product Service
- Shipping Service không đọc trực tiếp bảng Order
- Các service giao tiếp thông qua ID tham chiếu

Đây là tư duy thiết kế khá tốt trong kiến trúc phân tán.

Tài liệu cũng cho thấy sự chú trọng tới khả năng mở rộng trong tương lai, ví dụ:

- Có thể thêm loại sản phẩm mới
- Thêm phương thức thanh toán
- Mở rộng AI model mà không cần sửa toàn bộ hệ thống.

3. Thiết kế Database cho từng Service

Đây là một trong những phần được trình bày tốt nhất trong tài liệu.

Em đã xây dựng tương đối đầy đủ:

- Schema
- Field
- Relationship
- Trạng thái nghiệp vụ
- CREATE TABLE

cho từng service.

Một số điểm tích cực:

- Mỗi service có database riêng

- Sử dụng ID tham chiếu thay vì join trực tiếp
- Có snapshot dữ liệu đơn hàng
- Thiết kế trạng thái rõ ràng

Ví dụ:

- Payment có các trạng thái initiated, paid, failed, refunded
- Shipment có pending, shipping, delivered, cancelled

Điều này giúp mô hình hóa đúng vòng đời nghiệp vụ.

Ngoài ra, em còn thực hiện so sánh:

- MySQL
- PostgreSQL

và nêu được:

- PostgreSQL mạnh hơn ở JSON/JSONB
- MySQL phù hợp CRUD thông thường

Đây là nội dung có giá trị thực tế.

4. API Design

Phần API được trình bày khá rõ ràng:

- Endpoint
- HTTP method
- View class
- Chức năng

Ví dụ:

- GET /payments/
- POST /payments/create/
- PATCH /payments/<id>/status/

Điểm tích cực là em đã phân biệt được:

- API query
- API update trạng thái
- API theo customer
- API theo order

Điều này cho thấy hệ thống được suy nghĩ theo hướng service-oriented.

Tuy nhiên:

- Chưa có ví dụ request/response JSON
- Chưa mô tả error handling chi tiết
- Chưa có authentication flow cụ thể

Nhưng xét trong phạm vi môn học, mức độ hiện tại vẫn tương đối ổn.

5. UML và mô hình hóa hệ thống

Tài liệu đã có các phần:

- Class mapping
- Database mapping
- Luồng trạng thái

Tuy nhiên:

- Chưa có nhiều UML chuẩn
- Thiếu Use Case Diagram
- Thiếu Sequence Diagram
- Thiếu Activity Diagram

Đây là điểm còn hạn chế vì UML là phần quan trọng trong các môn phân tích và thiết kế hệ thống.

Dù vậy, em vẫn thể hiện được luồng xử lý thông qua mô tả nghiệp vụ và bảng trạng thái nên phần nào bù lại được thiếu sót này.

IV. ĐÁNH GIÁ CHƯƠNG 3 – AI SERVICE

1. Ý tưởng tích hợp AI vào E-Commerce

Đây là điểm tạo khác biệt cho bài tiểu luận.

Em không chỉ xây dựng hệ thống E-Commerce thông thường mà còn tích hợp thêm:

- Recommendation System
- Chatbot
- RAG
- Knowledge Graph

Điều này giúp tài liệu có tính hiện đại và phù hợp với xu hướng AI Application hiện nay.

Các mục tiêu AI được xác định khá rõ:

- Cá nhân hóa trải nghiệm
- Tăng conversion rate
- Tự động hóa tư vấn
- Khai phá dữ liệu người dùng

2. Kiến trúc AI Service

Tài liệu mô tả khá tốt pipeline AI:

- Data Lake
- Preprocessing
- Inference Engine
- Serving Layer

Việc tách AI thành một microservice riêng cũng là hướng tiếp cận hợp lý, giúp:

- Giảm tải transactional system
- Dễ nâng cấp model
- Triển khai độc lập

Đây là một điểm cộng đáng ghi nhận.

3. Mô hình LSTM

Em đã trình bày:

- Khái niệm sequence modeling
- Forget gate
- Input gate
- Output gate

và liên hệ với hành vi mua sắm của khách hàng.

Điểm tích cực là tài liệu không chỉ nêu định nghĩa mà còn cố gắng liên hệ với nghiệp vụ E-Commerce.

Tuy nhiên:

- Chưa có code huấn luyện
- Chưa có dataset minh họa
- Chưa có kết quả đánh giá model

Do đó phần AI hiện mới mạnh về mặt ý tưởng hơn là triển khai thực tế.

4. Knowledge Graph và Neo4j

Đây là phần tương đối nổi bật của tài liệu.

Em đã:

- Giới thiệu Property Graph
- Node
- Relationship
- Label
- Cypher query

và có ví dụ truy vấn recommendation.

Nội dung này thể hiện em có tìm hiểu thêm các công nghệ mới thay vì chỉ sử dụng relational database truyền thống.

Đặc biệt, việc áp dụng:

- Collaborative filtering
- Content-based recommendation

cho thấy hướng tiếp cận tương đối phù hợp với hệ thống gợi ý sản phẩm.

5. RAG và Chatbot

Tài liệu đã giới thiệu mô hình RAG và pipeline retrieval tương đối đúng định hướng.

Đây là nội dung hiện đại và có giá trị thực tế cao trong các hệ thống AI hiện nay.

Tuy nhiên:

- Chưa có vector database cụ thể
- Chưa có embedding model
- Chưa có triển khai LLM thật

Vì vậy phần này vẫn thiên về nghiên cứu và mô tả kiến trúc hơn là implementation.

V. ĐÁNH GIÁ CHƯƠNG 4 – TRIỂN KHAI HỆ THỐNG

1. Công nghệ sử dụng

Em đã lựa chọn các công nghệ phù hợp:

- Django
- PostgreSQL
- Docker
- Kubernetes
- Neo4j
- JWT

Đây đều là những công nghệ phổ biến và có khả năng triển khai thực tế.

Việc sử dụng Django cho Microservices là lựa chọn hợp lý trong môi trường học tập vì:

- Dễ phát triển nhanh
- Hệ sinh thái mạnh
- Có Django REST Framework

2. Triển khai và vận hành

Tài liệu có đề cập:

- Containerization
- API Gateway
- Logging
- Tracing
- Monitoring

Đây là điểm cộng vì nhiều bài tiểu luận thường bỏ qua phần vận hành hệ thống.

Em cũng nhấn mạnh:

- trace_id
- log tập trung
- khả năng debug request

Điều này cho thấy có tư duy DevOps và observability cơ bản.

3. Hạn chế của phần triển khai

Mặc dù có định hướng triển khai khá tốt, nhưng:

- Chưa có screenshot hệ thống
- Chưa có giao diện thực tế
- Chưa có benchmark
- Chưa có API response demo
- Chưa có kết quả AI trực quan

Đây là phần khiến tài liệu giảm bớt tính thực nghiệm.

Nếu bổ sung thêm:

- hình ảnh chạy Docker
- Postman testing
- chatbot demo

- giao diện frontend

thì bài tiểu luận sẽ thuyết phục hơn nhiều.

VI. ĐÁNH GIÁ TỔNG HỢP

Thứ nhất, tư duy kiến trúc tốt

Em hiểu khá đúng:

- Bounded Context
- Loose Coupling
- Distributed Monolith
- Database per service

Đây là điểm mạnh nổi bật của bài.

Thứ hai, thiết kế hệ thống tương đối đầy đủ

Các thành phần:

- Order
- Payment
- Shipment
- Cart
- Product

được mô hình hóa khá hợp lý.

Thứ ba, có sự cập nhật công nghệ hiện đại

Việc đưa thêm:

- AI Service
- Neo4j
- RAG
- Recommendation System

giúp tài liệu có chiều sâu và tính mới.

Thứ tư, có tư duy mở rộng hệ thống

Báo cáo đã quan tâm:

- scalability
- maintainability
- extensibility

thay vì chỉ tập trung vào chức năng đơn giản.

VII. NHỮNG ĐIỂM CẦN CẢI THIỆN

Bên cạnh các ưu điểm, tài liệu vẫn còn một số hạn chế cần cải thiện:

- Thiếu nhiều UML diagram chuẩn
- Chưa có nhiều hình minh họa trực quan
- Phần AI chưa có kết quả thực nghiệm rõ ràng
- Chưa có nhiều code triển khai thực tế
- Thiếu giao diện demo và benchmark

Tuy nhiên, các hạn chế này chủ yếu nằm ở mức độ triển khai sâu, không ảnh hưởng quá lớn tới định hướng và tư duy thiết kế tổng thể của bài tiểu luận.

VIII. KẾT LUẬN

Nhìn chung, đây là một bài tiểu luận có chất lượng tương đối khá, thể hiện được:

- Sự đầu tư nghiên cứu
- Tư duy thiết kế hệ thống
- Hiểu biết về Microservices
- Kiến thức nền tảng về AI Application

Bài làm có phạm vi rộng, tiếp cận nhiều công nghệ hiện đại và thể hiện được định hướng xây dựng hệ thống gần với thực tế.

Mặc dù vẫn còn một số điểm cần cải thiện về:

- trực quan hóa hệ thống,
- triển khai thực nghiệm,

- UML,
- và minh chứng kết quả,

nhưng xét trong phạm vi học phần, tài liệu vẫn đáp ứng tương đối tốt các yêu cầu về:

- phân tích,
- thiết kế,
- mô hình hóa,
- và định hướng triển khai hệ thống phần mềm hiện đại.

Đặc biệt, phần tư duy kiến trúc và phân rã Microservices là điểm đáng ghi nhận của bài tiểu luận.

SOURCE CODE:

<https://github.com/lamlocan5/Software-Architecture-And-Design>

PTTT